

Finding Small Steps for Hard Questions, in work on the Process Semantics of Regular Expressions

Clemens Grabmayer

<https://clegra.github.io>

Department of Computer Science

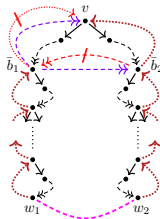
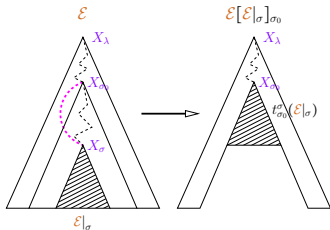


L'Aquila, Italy

IFIP 1.6 Working Group Meeting

Lisbon

July 18, 2026



Overview

- ▶ Regular expressions (also: 1-free/under-star-1-free $(*/\perp)$ expr.s)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility questions)
 - ▶ axioms for $\llbracket \cdot \rrbracket_P$ -identity ($\leadsto$ completeness question)
- ▶ **Small steps** for **hard questions**
 - (st1) reduction steps for well-behaved graphs under bisimilarity
 - ▶ $\llbracket \cdot \rrbracket_P$ -expressibility is decidable
 - (st2) collapse steps for LLEE graphs under bisimilarity
 - ▶ completeness for $(*/\perp)$ expressions
 - (st3) collapse steps for LLEE 1-graphs under bisimilarity, as far as possible
 - ▶ counterexample to collapse, but leads to completeness
- ▶ Interpretation–readback **correspondences**
 - (irc) LLEE/1-LLEE graphs $\triangleq$ image of $P^\bullet$ restricted to $(*/\perp)$ /all expr.s
 - ▶ where $P^\bullet$ a compact refinement of P
 - ▶ crucial for P -/ $\llbracket \cdot \rrbracket_P$ -expressibility, and completeness questions
- ▶ Outlook

Overview

- ▶ Regular expressions (also: 1-free/[under-star-1-free](#) $(*/\perp)$ expr.s)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ [expressibility questions](#))
 - ▶ axioms for $\llbracket \cdot \rrbracket_P$ -identity ($\leadsto$ [completeness question](#))
- ▶ **Small steps** for **hard questions**
 - (st1) **reduction steps** for **well-behaved** graphs under bisimilarity
 - ▶ $\llbracket \cdot \rrbracket_P$ -**expressibility** is decidable
 - (st2) **collapse steps** for **LLEE** graphs under bisimilarity
 - ▶ **completeness** for $(*/\perp)$ expressions
 - (st3) **collapse steps** for **LLEE** 1-graphs under bisimilarity, as far as possible
 - ▶ counterexample to collapse, but leads to **completeness**
- ▶ Interpretation–readback **correspondences**
 - (irc) **LLEE/1-LLEE** graphs $\triangleq$ image of $P^\bullet$ restricted to $(*/\perp)$ /all expr.s
 - ▶ where $P^\bullet$ a **compact** refinement of P
 - ▶ crucial for P -/ $\llbracket \cdot \rrbracket_P$ -**expressibility**, and **completeness** questions
- ▶ Outlook

Overview

- ▶ Regular expressions (also: 1-free/[under-star-1-free](#) $(*/\perp)$ expr.s)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ [expressibility questions](#))
 - ▶ axioms for $\llbracket \cdot \rrbracket_P$ -identity ($\leadsto$ [completeness question](#))
- ▶ [Small steps](#) for [hard questions](#)
 - (st1) [reduction steps](#) for [well-behaved](#) graphs under bisimilarity
 - ▶ $\llbracket \cdot \rrbracket_P$ -[expressibility](#) is decidable
 - (st2) [collapse steps](#) for [LLEE](#) graphs under bisimilarity
 - ▶ [completeness](#) for $(*/\perp)$ expressions
 - (st3) [collapse steps](#) for [LLEE](#) 1-graphs under bisimilarity, [as far as possible](#)
 - ▶ counterexample to collapse, but leads to [completeness](#)
- ▶ Interpretation–readback [correspondences](#)
 - (irc) [LLEE/1-LLEE](#) graphs $\triangleq$ image of $P^\bullet$ restricted to $(*/\perp)$ /all expr.s
 - ▶ where $P^\bullet$ a [compact](#) refinement of P
 - ▶ crucial for P -/ $\llbracket \cdot \rrbracket_P$ -[expressibility](#), and [completeness](#) questions
- ▶ Outlook

Overview

- ▶ Regular expressions (also: 1-free/[under-star-1-free](#) $(*/\perp)$ expr.s)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ [expressibility questions](#))
 - ▶ axioms for $\llbracket \cdot \rrbracket_P$ -identity ($\leadsto$ [completeness question](#))
- ▶ [Small steps](#) for [hard questions](#)
 - (st1) [reduction steps](#) for [well-behaved](#) graphs under bisimilarity
 - ▶ $\llbracket \cdot \rrbracket_P$ -[expressibility](#) is decidable
 - (st2) [collapse steps](#) for LLEE graphs under bisimilarity
 - ▶ [completeness](#) for $(*/\perp)$ expressions
 - (st3) [collapse steps](#) for LLEE 1-graphs under bisimilarity, [as far as possible](#)
 - ▶ counterexample to collapse, but leads to [completeness](#)
- ▶ Interpretation–readback [correspondences](#)
 - (irc) LLEE/1-LLEE graphs $\hat{=}$ image of $P^\bullet$ restricted to $(*/\perp)$ /all expr.s
 - ▶ where $P^\bullet$ a [compact](#) refinement of P
 - ▶ crucial for P -/ $\llbracket \cdot \rrbracket_P$ -[expressibility](#), and [completeness](#) questions
- ▶ Outlook

Regular Expression and Milner's Process Interpretation

- ▶ Regular expressions (also: 1-free/under-star-1-free $(*/\perp)$ expr.s)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility questions)
 - ▶ axioms for $\llbracket \cdot \rrbracket_P$ -identity ($\leadsto$ completeness question)
- ▶ Small steps for hard questions
 - (st1) reduction steps for well-behaved graphs under bisimilarity
 - ▶ $\llbracket \cdot \rrbracket_P$ -expressibility is decidable
 - (st2) collapse steps for LLEE graphs under bisimilarity
 - ▶ completeness for $(*/\perp)$ expressions
 - (st3) collapse steps for LLEE 1-graphs under bisimilarity, as far as possible
 - ▶ counterexample to collapse, but leads to completeness
- ▶ Interpretation–readback correspondences
 - (irc) LLEE/1-LLEE graphs $\triangleq$ image of $P^\bullet$ restricted to $(*/\perp)$ /all expr.s
 - ▶ where $P^\bullet$ a compact refinement of P
 - ▶ crucial for P -/ $\llbracket \cdot \rrbracket_P$ -expressibility, and completeness questions
- ▶ Outlook

Process interpretation P of regular expressions (Milner, 1984)

$0 \xrightarrow{P} \text{deadlock } \delta, \text{ no termination}$

$a \xrightarrow{P} \text{atomic action } a, \text{ then terminate}$

Process interpretation P of regular expressions (Milner, 1984)

$0 \xrightarrow{P} \text{deadlock } \delta, \text{ no termination}$

$a \xrightarrow{P} \text{atomic action } a, \text{ then terminate}$

$e_1 + e_2 \xrightarrow{P} (\text{choice}) \text{ execute } P(e_1) \text{ or } P(e_2)$

$e_1 \cdot e_2 \xrightarrow{P} (\text{sequentialization}) \text{ execute } P(e_1), \text{ then } P(e_2)$

$e^* \xrightarrow{P} (\text{iteration}) \text{ repeat (terminate or execute } P(e))$

Process interpretation P of regular expressions (Milner, 1984)

$0 \xrightarrow{P}$ deadlock δ , no termination

$0^* \xrightarrow{P}$ empty-step process ϵ , then terminate

$a \xrightarrow{P}$ atomic action a , then terminate

$e_1 + e_2 \xrightarrow{P}$ (*choice*) execute $P(e_1)$ or $P(e_2)$

$e_1 \cdot e_2 \xrightarrow{P}$ (*sequentialization*) execute $P(e_1)$, then $P(e_2)$

$e^* \xrightarrow{P}$ (*iteration*) repeat (terminate or execute $P(e)$)

Process interpretation P of regular expressions (Milner, 1984)

$0 \xrightarrow{P}$ deadlock δ , no termination

$1 \xrightarrow{P}$ empty-step process ϵ , then terminate

$a \xrightarrow{P}$ atomic action a , then terminate

$e_1 + e_2 \xrightarrow{P}$ (*choice*) execute $P(e_1)$ or $P(e_2)$

$e_1 \cdot e_2 \xrightarrow{P}$ (*sequentialization*) execute $P(e_1)$, then $P(e_2)$

$e^* \xrightarrow{P}$ (*iteration*) repeat (terminate or execute $P(e)$)

Regular Expressions

Definition (*~ Copi-Elgot-Wright, 1958*)

Regular expressions over alphabet A with unary Kleene star:

$e, e_1, e_2 ::= 0 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$

- ▶ symbol 0 instead of $\emptyset$

Regular Expressions

Definition (~ *Copi-Elgot-Wright, 1958*)

Regular expressions over alphabet A with unary Kleene star:

$e, e_1, e_2 ::= 0 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star * : **1** is definable as **0^{*}**

Regular Expressions

Definition ($\sim$ Kleene, 1951, $\sim$ Copi-Elgot-Wright, 1958)

Regular expressions over alphabet A with unary / binary Kleene star:

$$e, e_1, e_2 ::= 0 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$$

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e_1^{\oplus} e_2 \quad (\text{for } a \in A).$$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star * : **1** is definable as **0** *
- ▶ with binary star $^{\oplus}$: **1** is **not** definable (in its absence)

Regular Expressions

Definition ($\sim$ Kleene, 1951, $\sim$ Copi-Elgot-Wright, 1958)

Regular expressions over alphabet A with unary / binary Kleene star:

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$$

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e_1^{\oplus} e_2 \quad (\text{for } a \in A).$$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star * : **1** is definable as **0^{*}**
- ▶ with binary star $^{\oplus}$: **1** is **not** definable (in its absence)

Regular Expressions (1-free)

Definition ($\sim$ Kleene, 1951, $\sim$ Copi-Elgot-Wright, 1958)

Regular expressions over alphabet A with unary / binary Kleene star:

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$$

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e_1^{\oplus} e_2 \quad (\text{for } a \in A).$$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star * : **1** is definable as **0^{*}**
- ▶ with binary star $^{\oplus}$: **1** is **not** definable (in its absence)

Definition (for process interpretation)

1-free regular expressions over alphabet A with binary Kleene star:

$$f, f_1, f_2 ::= 0 \mid a \mid f_1 + f_2 \mid f_1 \cdot f_2 \mid f_1^{\oplus} f_2 \quad (\text{for } a \in A).$$

Regular Expressions (1-free)

Definition ($\sim$ Kleene, 1951, $\sim$ Copi-Elgot-Wright, 1958)

Regular expressions over alphabet A with unary / binary Kleene star:

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$$

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e_1^{\otimes} e_2 \quad (\text{for } a \in A).$$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star * : **1** is definable as **0^{*}**
- ▶ with binary star $^{\otimes}$: **1** is **not** definable (in its absence)

Definition (for process interpretation)

1-free regular expressions over alphabet A with unary / binary Kleene star:

$$f, f_1, f_2 ::= 0 \mid a \mid f_1 + f_2 \mid f_1 \cdot f_2 \mid (f_1^*) \cdot f_2 \quad (\text{for } a \in A),$$

$$f, f_1, f_2 ::= 0 \mid a \mid f_1 + f_2 \mid f_1 \cdot f_2 \mid f_1^{\otimes} f_2 \quad (\text{for } a \in A).$$

Regular Expressions (under-star-/1-free)

Definition ($\sim$ Kleene, 1951, $\sim$ Copi-Elgot-Wright, 1958)

Regular expressions over alphabet A with unary / binary Kleene star:

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$$

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e_1^{\oplus} e_2 \quad (\text{for } a \in A).$$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star * : **1** is definable as **0** *
- ▶ with binary star $^{\oplus}$: **1** is **not** definable (in its absence)

Definition (for process interpretation)

The set $RExp^{(+)}(A)$ of **1-free** regular expressions over A is defined by:

$$f, f_1, f_2 ::=_{\text{assoc}(\cdot)} 0 \mid a \mid f_1 + f_2 \mid f_1 \cdot f_2 \mid f_1^* \cdot f_2 \quad (\text{for } a \in A),$$

Regular Expressions $((*/\perp)/1\text{-free})$

Definition ($\sim$ Kleene, 1951, $\sim$ Copi-Elgot-Wright, 1958)

Regular expressions over alphabet A with unary / binary Kleene star:

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$$

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e_1^{\oplus} e_2 \quad (\text{for } a \in A).$$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star $*$: **1** is definable as **0** $*$
- ▶ with binary star $\oplus$: **1** is **not** definable (in its absence)

Definition (for process interpretation)

The set $RExp^{(+)}(A)$ of **1-free regular expressions** over A is defined by:

$$f, f_1, f_2 ::=_{\text{assoc}(\cdot)} 0 \mid a \mid f_1 + f_2 \mid f_1 \cdot f_2 \mid f_1^* \cdot f_2 \quad (\text{for } a \in A),$$

the set $RExp^{(*\perp)}(A)$ of **under-star-1-free $((*/\perp)$ reg. expr.** over A by:

$$uf, uf_1, uf_2 ::= 0 \mid 1 \mid a \mid uf_1 + uf_2 \mid uf_1 \cdot uf_2 \mid f^* \quad (\text{for } a \in A).$$

Process semantics $\llbracket \cdot \rrbracket_P$ of regular expressions (Milner, 1984)

$0 \xrightarrow{P}$ deadlock δ , no termination

$1 \xrightarrow{P}$ empty-step process ϵ , then terminate

$a \xrightarrow{P}$ atomic action a , then terminate

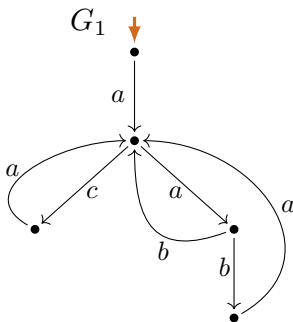
$e_1 + e_2 \xrightarrow{P}$ (*choice*) execute $P(e_1)$ or $P(e_2)$

$e_1 \cdot e_2 \xrightarrow{P}$ (*sequentialization*) execute $P(e_1)$, then $P(e_2)$

$e^* \xrightarrow{P}$ (*iteration*) repeat (terminate or execute $P(e)$)

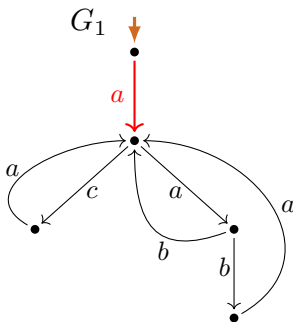
$\llbracket e \rrbracket_P := \llbracket P(e) \rrbracket_{\leftrightarrow}$ (bisimilarity equivalence class of process $P(e)$)

P -expressibility and $[[\cdot]]_P$ -expressibility (example, informally)



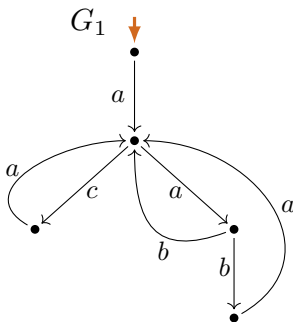
$$P\left(\overbrace{a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0}^f \right)$$

P -expressibility and $[[\cdot]]_P$ -expressibility (example, informally)



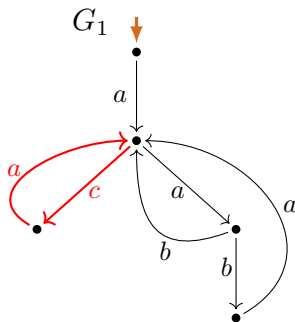
$$P\left(\overbrace{a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0}^f \right)$$

P -expressibility and $[[\cdot]]_P$ -expressibility (example, informally)



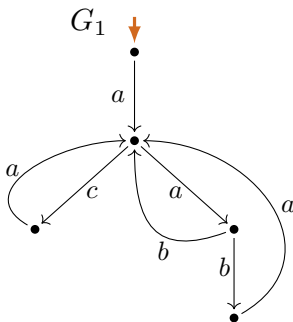
$$P\left(\overbrace{a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0}^f \right)$$

P -expressibility and $[[\cdot]]_P$ -expressibility (example, informally)



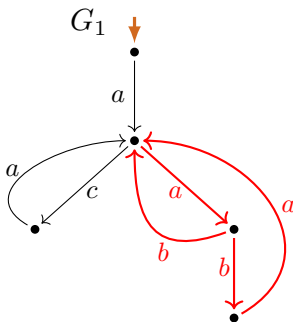
$$P\left(\overbrace{a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0}^f \right)$$

P -expressibility and $[[\cdot]]_P$ -expressibility (example, informally)



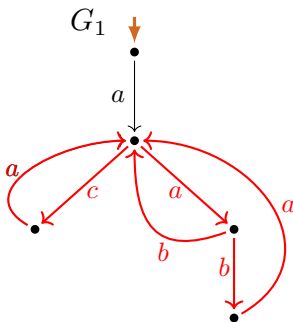
$$P\left(\overbrace{a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0}^f \right)$$

P -expressibility and $[[\cdot]]_P$ -expressibility (example, informally)



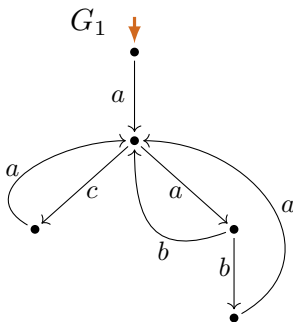
$$P\left(\overbrace{a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0}^f \right)$$

P -expressibility and $[[\cdot]]_P$ -expressibility (example, informally)



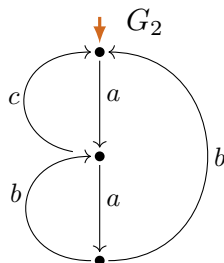
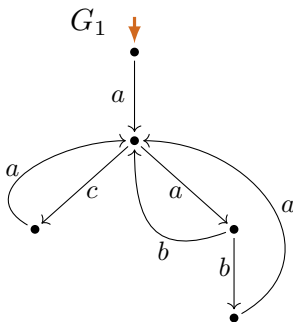
$$P\left(\overbrace{a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0}^f \right)$$

P -expressibility and $[[\cdot]]_P$ -expressibility (example, informally)



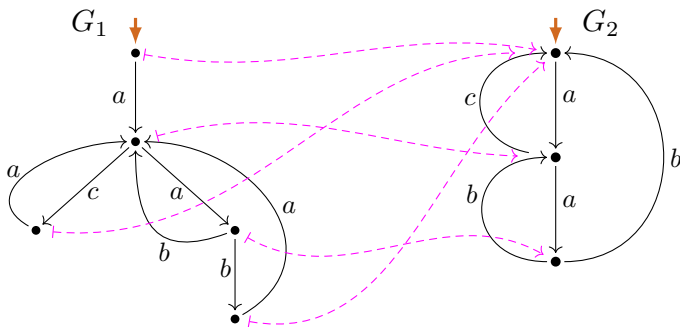
$$P\left(\overbrace{a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0}^f \right)$$

P -expressibility and $[[\cdot]]_P$ -expressibility (example, informally)



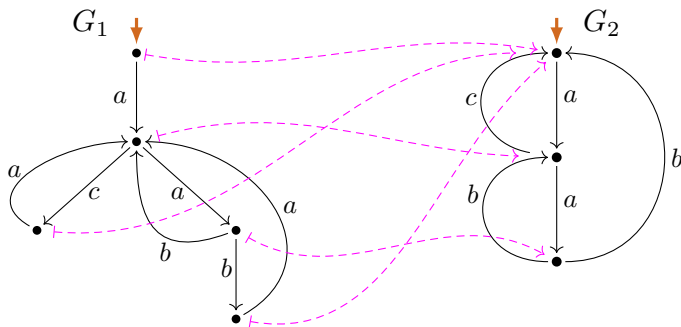
$$P\left(\overbrace{a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0}^f \right)$$

P -expressibility and $[[\cdot]]_P$ -expressibility (example, informally)



$$P\left(\overbrace{a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0}^f\right)$$

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility (example, informally)



$$P\left(\overbrace{a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0}^f\right)$$

$$G_1 \in \llbracket f \rrbracket_P$$

$$G_2 \in \llbracket f \rrbracket_P$$

Process interpretation P (formally)

Definition (Transition system specification $\mathcal{T}$)

$$\frac{}{a \xrightarrow{a} 1} \quad \frac{e_i \xrightarrow{a} e'_i}{e_1 + e_2 \xrightarrow{a} e'_i} \quad (i \in \{1, 2\})$$

Process interpretation P (formally)

Definition (Transition system specification $\mathcal{T}$)

$$\begin{array}{c}
 \frac{}{a \xrightarrow{a} 1} \qquad \frac{e_i \xrightarrow{a} e'_i}{e_1 + e_2 \xrightarrow{a} e'_i} \quad (i \in \{1, 2\}) \\
 \\
 \frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1 \cdot e_2} \qquad \frac{e \xrightarrow{a} e'}{e^* \xrightarrow{a} e' \cdot e^*}
 \end{array}$$

Process interpretation P (formally)

Definition (Transition system specification $\mathcal{T}$)

$$\begin{array}{c}
 \frac{}{1 \Downarrow} \qquad \frac{e_i \Downarrow}{(e_1 + e_2) \Downarrow} \ (i \in \{1, 2\}) \qquad \frac{e_1 \Downarrow \quad e_2 \Downarrow}{(e_1 \cdot e_2) \Downarrow} \qquad \frac{}{(e^*) \Downarrow} \\
 \\
 \frac{}{a \xrightarrow{a} 1} \qquad \frac{e_i \xrightarrow{a} e'_i}{e_1 + e_2 \xrightarrow{a} e'_i} \ (i \in \{1, 2\}) \\
 \\
 \frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1 \cdot e_2} \qquad \frac{e \xrightarrow{a} e'}{e^* \xrightarrow{a} e' \cdot e^*}
 \end{array}$$

Process interpretation P (formally)

Definition (Transition system specification $\mathcal{T}$)

$$\begin{array}{c}
 \frac{}{1 \Downarrow} \qquad \frac{e_i \Downarrow}{(e_1 + e_2) \Downarrow} \ (i \in \{1, 2\}) \qquad \frac{e_1 \Downarrow \quad e_2 \Downarrow}{(e_1 \cdot e_2) \Downarrow} \qquad \frac{}{(e^*) \Downarrow} \\
 \\
 \frac{}{a \xrightarrow{a} 1} \qquad \frac{e_i \xrightarrow{a} e'_i}{e_1 + e_2 \xrightarrow{a} e'_i} \ (i \in \{1, 2\}) \\
 \\
 \frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1 \cdot e_2} \qquad \frac{e_1 \Downarrow \quad e_2 \xrightarrow{a} e'_2}{e_1 \cdot e_2 \xrightarrow{a} e'_2} \qquad \frac{e \xrightarrow{a} e'}{e^* \xrightarrow{a} e' \cdot e^*}
 \end{array}$$

Process interpretation P (formally)

Definition (Transition system specification $\mathcal{T}$)

$$\begin{array}{c}
 \frac{}{1 \Downarrow} \quad \frac{e_i \Downarrow}{(e_1 + e_2) \Downarrow} \ (i \in \{1, 2\}) \quad \frac{e_1 \Downarrow \quad e_2 \Downarrow}{(e_1 \cdot e_2) \Downarrow} \quad \frac{}{(e^*) \Downarrow} \\
 \\
 \frac{}{a \xrightarrow{a} 1} \quad \frac{e_i \xrightarrow{a} e'_i}{e_1 + e_2 \xrightarrow{a} e'_i} \ (i \in \{1, 2\}) \\
 \\
 \frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1 \cdot e_2} \quad \frac{e_1 \Downarrow \quad e_2 \xrightarrow{a} e'_2}{e_1 \cdot e_2 \xrightarrow{a} e'_2} \quad \frac{e \xrightarrow{a} e'}{e^* \xrightarrow{a} e' \cdot e^*}
 \end{array}$$

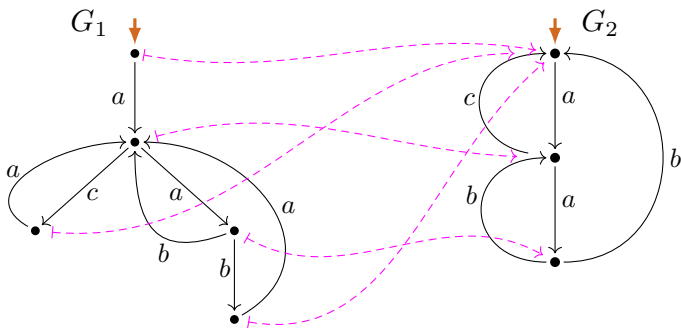
Definition

The **process interpretation** $P(e)$ of a regular expression e :

$P(e) :=$ **labeled transition graph** generated by e by derivations in $\mathcal{T}$,

The **process semantics** $\llbracket e \rrbracket_P$ of e is: $\llbracket e \rrbracket_P := [P(e)]_{\leftrightarrow}$.

P -expressibility and $[[\cdot]]_P$ -expressibility (example, informally)

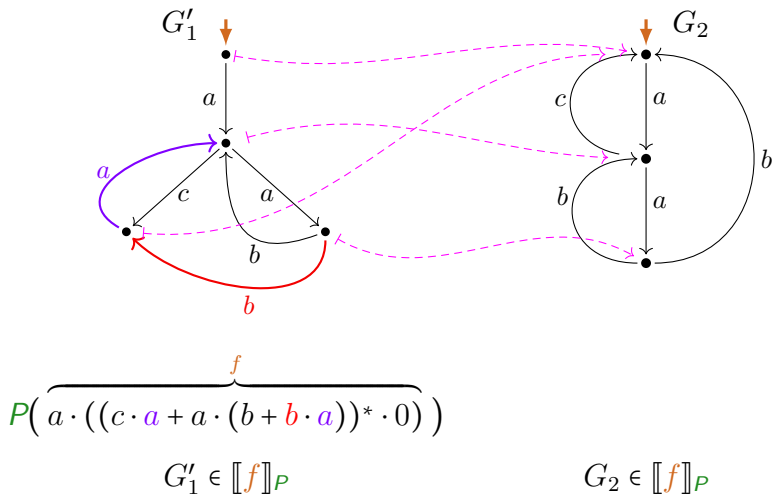


$$P\left(\overbrace{a \cdot ((c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0)}^f\right)$$

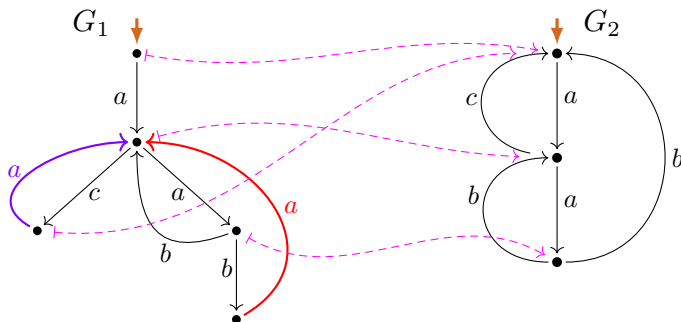
$$G_1 \in [[f]]_P$$

$$G_2 \in [[f]]_P$$

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility (example, formally)



P -expressibility and $[[\cdot]]_P$ -expressibility (example, formally)

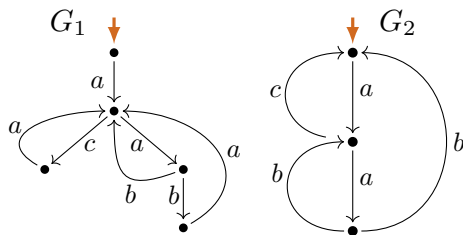


$$P\left(\overbrace{a \cdot ((c \cdot a + a \cdot (b + b \cdot (a + a)))^* \cdot 0)}^f\right)$$

$$G_1 \in [[f]]_P$$

$$G_2 \in [[f]]_P$$

P -expressibility and $[[\cdot]]_P$ -expressibility (examples)

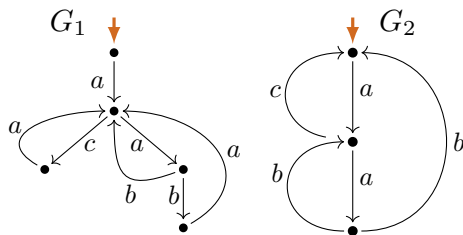


P -expressible

$[[\cdot]]_P$ -expressible

$[[\cdot]]_P$ -expressible

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility (examples)



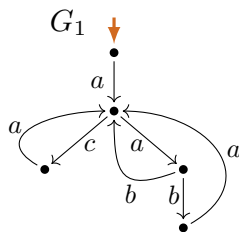
P -expressible

?

$\llbracket \cdot \rrbracket_P$ -expressible

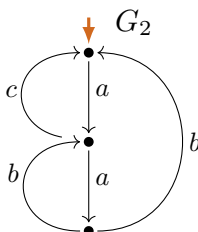
$\llbracket \cdot \rrbracket_P$ -expressible

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility (examples)



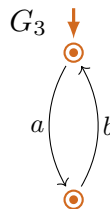
P -expressible

$\llbracket \cdot \rrbracket_P$ -expressible



?

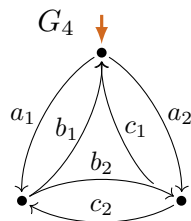
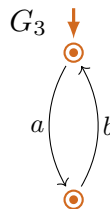
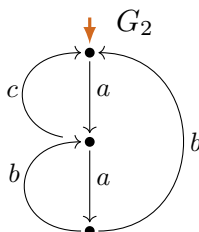
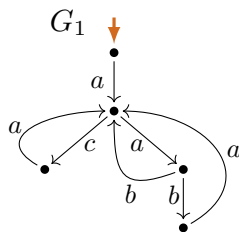
$\llbracket \cdot \rrbracket_P$ -expressible



not P -expressible

not $\llbracket \cdot \rrbracket_P$ -expressible

P -expressibility and $[[\cdot]]_P$ -expressibility (examples)



P -expressible

?

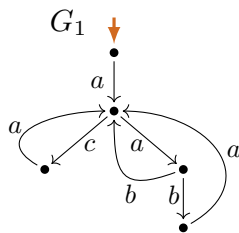
not P -expressible

$[[\cdot]]_P$ -expressible

$[[\cdot]]_P$ -expressible

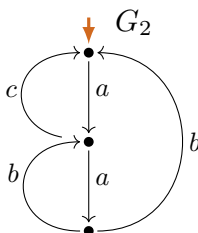
not $[[\cdot]]_P$ -expressible

P -expressibility and $[[\cdot]]_P$ -expressibility (examples)



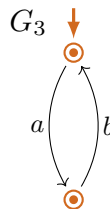
P -expressible

$[[\cdot]]_P$ -expressible



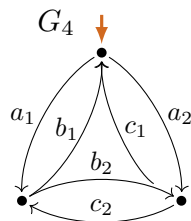
?

$[[\cdot]]_P$ -expressible



not P -expressible

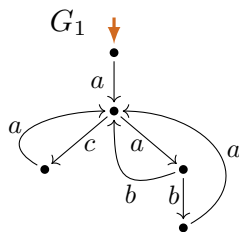
not $[[\cdot]]_P$ -expressible



Expressibility Questions:

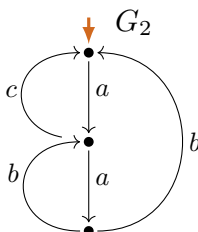
How can P -expressibility and $[[\cdot]]_P$ -expressibility be characterised?

P -expressibility and $[[\cdot]]_P$ -expressibility (examples)



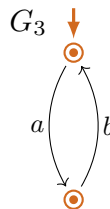
P -expressible

$[[\cdot]]_P$ -expressible



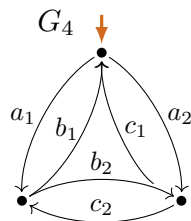
?

$[[\cdot]]_P$ -expressible



not P -expressible

not $[[\cdot]]_P$ -expressible

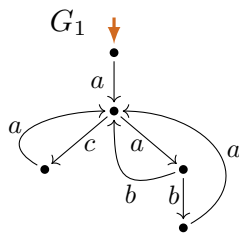


Expressibility Questions:

What structural property of finite charts [process graphs]

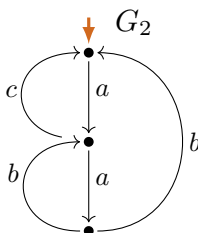
is necessary and sufficient of star behaviour $[[\cdot]]_P$ -expressibility (Milner, 1984)

P -expressibility and $[[\cdot]]_P$ -expressibility (examples)



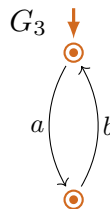
P -expressible

$[[\cdot]]_P$ -expressible



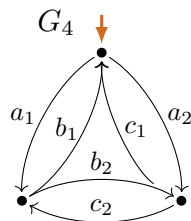
?

$[[\cdot]]_P$ -expressible



not P -expressible

not $[[\cdot]]_P$ -expressible



Expressibility Questions:

How can P -expressibility and $[[\cdot]]_P$ -expressibility be characterised?

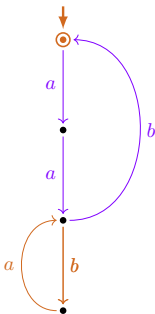
Overview

- ▶ Regular expressions (also: 1-free/under-star-1-free $(*/\perp)$ expr.s)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility questions)
 - ▶ axioms for $\llbracket \cdot \rrbracket_P$ -identity ($\leadsto$ completeness question)
- ▶ **Small steps** for **hard questions**
 - (st1) **reduction steps** for **well-behaved** graphs under bisimilarity
 - ▶ $\llbracket \cdot \rrbracket_P$ -**expressibility** is decidable
 - (st2) **collapse steps** for **LLEE** graphs under bisimilarity
 - ▶ **completeness** for $(*/\perp)$ expressions
 - (st3) **collapse steps** for **LLEE 1**-graphs under bisimilarity, **as far as possible**
 - ▶ counterexample to collapse, but leads to **completeness**
- ▶ Interpretation–readback **correspondences**
 - (irc) **LLEE/1-LLEE** graphs $\triangleq$ image of $P^\bullet$ restricted to $(*/\perp)$ /all expr.s
 - ▶ where $P^\bullet$ a **compact** refinement of P
 - ▶ crucial for P -/ $\llbracket \cdot \rrbracket_P$ -**expressibility**, and **completeness** questions
- ▶ Outlook

Small steps for hard questions

- ▶ Regular expressions (also: 1-free/under-star-1-free $(*/\perp)$ expr.s)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility questions)
 - ▶ axioms for $\llbracket \cdot \rrbracket_P$ -identity ($\leadsto$ completeness question)
- ▶ **Small steps** for **hard questions**
 - (st1) **reduction steps** for **well-behaved** graphs under bisimilarity
 - ▶ $\llbracket \cdot \rrbracket_P$ -**expressibility** is decidable
 - (st2) **collapse steps** for LLEE graphs under bisimilarity
 - ▶ completeness for $(*/\perp)$ expressions
 - (st3) **collapse steps** for LLEE 1-graphs under bisimilarity, as far as possible
 - ▶ counterexample to collapse, but leads to completeness
- ▶ Interpretation–readback **correspondences**
 - (irc) LLEE/1-LLEE graphs $\hat{=}$ image of $P^\bullet$ restricted to $(*/\perp)$ /all expr.s
 - ▶ where $P^\bullet$ a **compact** refinement of P
 - ▶ crucial for P -/ $\llbracket \cdot \rrbracket_P$ -**expressibility**, and completeness questions
- ▶ Outlook

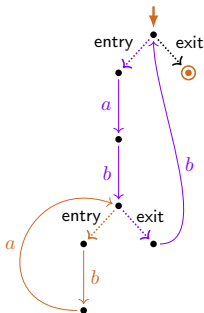
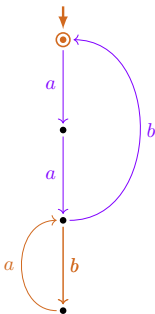
Well-behaved 1-graphs (with 1-transitions)



$$\sim P((aa(ba)^*b)^*)$$

$$P(1 \cdot (a \cdot ((a \cdot (b \cdot a)^*) \cdot b))^*)$$

Well-behaved 1-graphs (with 1-transitions)



well-behaved
loop-entry/-exit palm tree
(Corradini, Baeten)

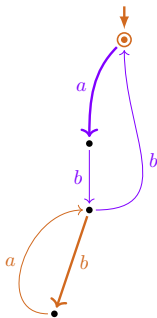
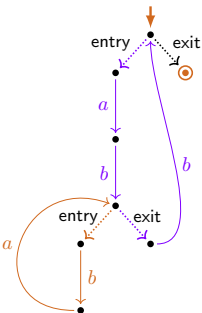
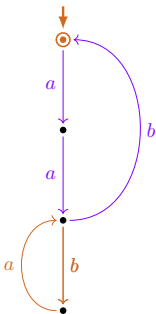
$$\sim P((aa(ba)^*b)^*)$$

$$\sim P((1 \cdot aa(1 \cdot ba)^* 1 \cdot b)^* (1 \cdot 1))$$

$$P(1 \cdot (a \cdot ((a \cdot (b \cdot a)^* \cdot b))^*))$$

$$P(1 \cdot (((1 \cdot a) \cdot (((a \cdot ((1 \cdot b) \cdot a)^* \cdot 1) \cdot b))^*) \cdot 1))$$

Well-behaved 1-graphs (with 1-transitions)



well-behaved
loop-entry/-exit palm tree
(Corradini, Baeten)

$$\sim P((aa(ba)^*b)^*)$$

$$\sim P((1aa(1ba)^*1b)^*(1.1))$$

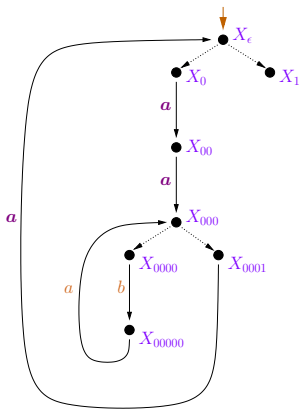
$$\sim P((aa(ba)^*b)^*)$$

$$P(1.(a.((a.(b.a)^*).b))^*)$$

$$P(1.((((1.a).(((a.((1.b).a)^*).1).b))^*).1))$$

$$P(1.(a.((a.(b.a)^*).b))^*)$$

Well-Behaved specification (example)



$$X_\epsilon = 1 \cdot X_0 + 1 \cdot X_1$$

$$X_0 = a \cdot X_{00}$$

$$X_{00} = a \cdot X_{000}$$

$$X_{000} = 1 \cdot X_{0000} + 1 \cdot X_{0001}$$

$$X_{0000} = b \cdot X_{00000}$$

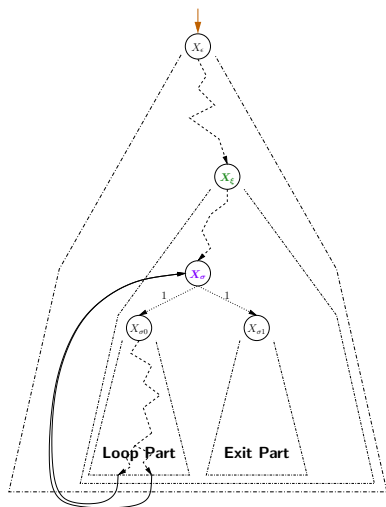
$$X_{00000} = a \cdot X_{000}$$

$$X_{0001} = a \cdot X_\epsilon$$

$$X_1 = 0$$

$$P((aa(ba)^*a)^*.0)$$

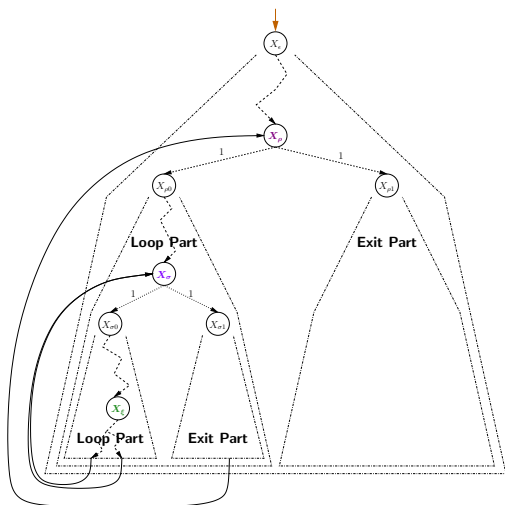
Well-Behaved specifications (well-beh./cycling variables)



$X_\xi, X_\epsilon \dots$ *well-behaved variables*
 (X_ξ “does not return” to a
 recursion variable above itself)

X_σ is a *cycling variable*
 (some recursion variable below X_σ
 “returns to” X_σ)

Well-Behaved specifications (cycles back to)



$X_\sigma, X_\rho \dots$ cycling variables

X_ξ cycles back to X_σ

(The nearest return of X_ξ
to a rec.var. above is to X_σ)

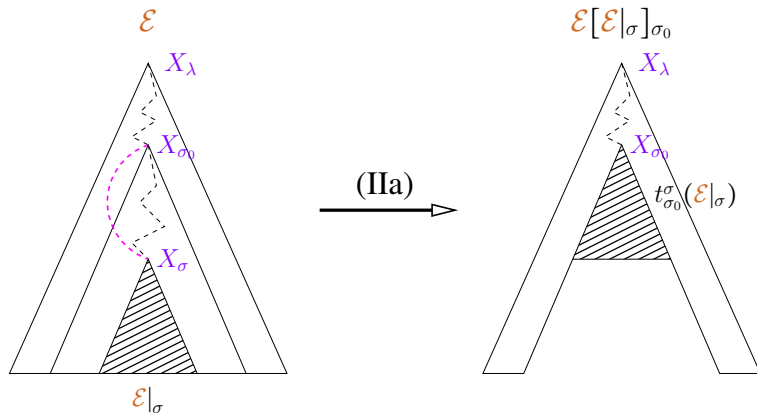
X_σ cycles back to X_ρ

$\llbracket \cdot \rrbracket_P$ -expressible = bisimilar to well-behaved

Theorem (Baeten/Corradini, 2005)

- (i) A graph G is $\llbracket \cdot \rrbracket_P$ -expressible
 $\iff G$ is the solution of a *well-behaved* specification.
- (ii) A graph G is $\llbracket \cdot \rrbracket_P$ -expressible
 $\iff$ there is a *well-behaved* graph G_{wb} such that $G \leftrightarrow G_{wb}$.

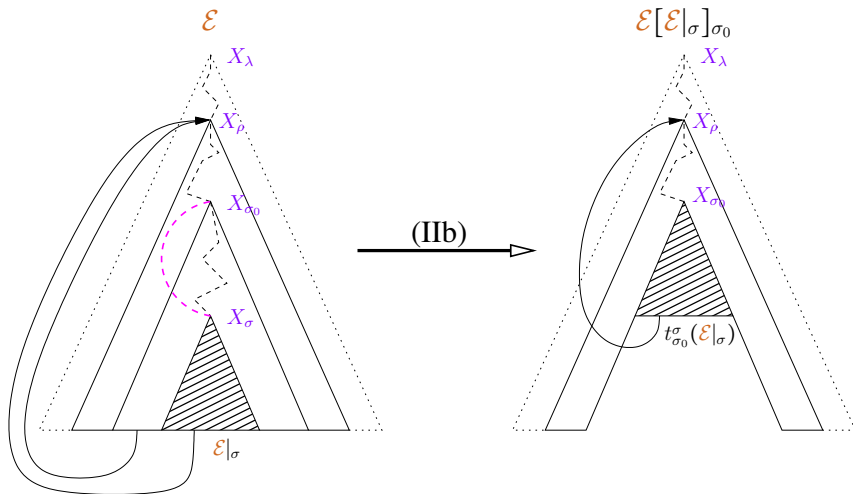
Reduction steps (IIa) for well-behaved 1-graphs



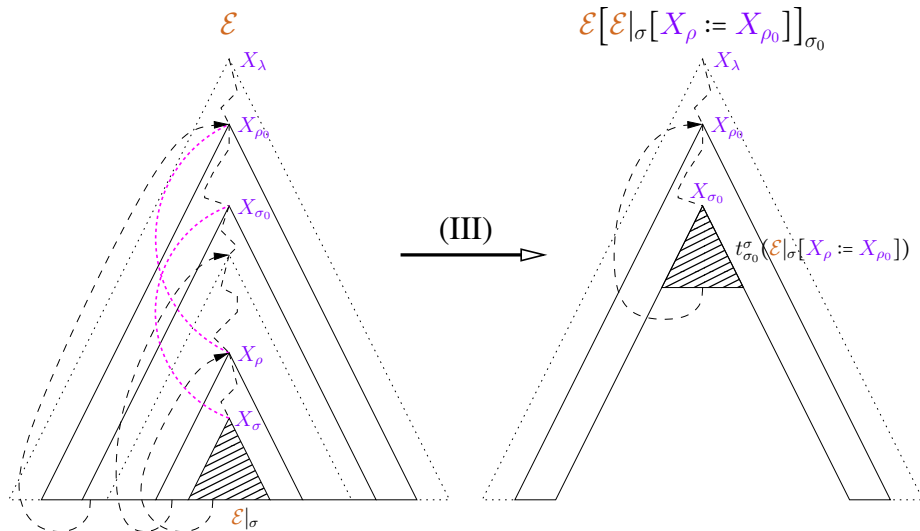
$$\langle X_\sigma | \mathcal{E} \rangle \Leftrightarrow \langle X_{\sigma_0} | \mathcal{E} \rangle$$

X_σ, X_{σ_0} are well-behaved

Reduction steps (IIb) for well-behaved 1-graphs



Reduction steps (III) for well-behaved 1-graphs



Result facilitated by steps (st1)

Theorem (*Baeten–Corradini–G, 2005*)

$[[\cdot]]_P$ -Expressibility of graphs is *decidable* (at least doubly-exponentially).

Result facilitated by steps (st1)

Reduction Lemma for well-behaved 1-graphs (*G, 2005*)

Let G be graph with n states and max. out-degree k . Suppose that $\underline{G}_{wb}$ is a well-behaved 1-graph with $\underline{G}_{wb} \Leftrightarrow G$ ($\Rightarrow G, \underline{G}_{wb}$ are $\llbracket \cdot \rrbracket_P$ -expressible).

Then $\underline{G}_{wb}$ can be reduced under bisimilarity by reduction steps (I)–(III) to a well-behaved 1-graph $\underline{G}_{wb}^{(0)}$ with:

- ▶ $\underline{G}_{wb}^{(0)} \Leftrightarrow \underline{G}_{wb} (\Leftrightarrow G),$
- ▶ $\text{depth} \leq (n+1)^3 \cdot 2^{3k},$ and max. out-degree $\leq k.$

Theorem (*Baeten–Corradini–G, 2005*)

$\llbracket \cdot \rrbracket_P$ -Expressibility of graphs is **decidable** (at least doubly-exponentially).

Result facilitated by steps (st1)

Reduction Lemma for well-behaved 1-graphs (*G, 2005*)

Let G be graph with n states and max. out-degree k . Suppose that $\underline{G}_{wb}$ is a well-behaved 1-graph with $\underline{G}_{wb} \Leftrightarrow G$ ($\Rightarrow G, \underline{G}_{wb}$ are $\llbracket \cdot \rrbracket_P$ -expressible). Then $\underline{G}_{wb}$ can be reduced under bisimilarity by reduction steps (I)–(III) to a well-behaved 1-graph $\underline{G}_{wb}^{(0)}$ with:

- ▶ $\underline{G}_{wb}^{(0)} \Leftrightarrow \underline{G}_{wb}$ ($\Leftrightarrow G$),
- ▶ $\text{depth} \leq (n+1)^3 \cdot 2^{3k}$, and max. out-degree $\leq k$.

Effective Expressibility Lemma (*G, 2005*)

A graph G with n states and max. out-degree k is $\llbracket \cdot \rrbracket_P$ -expressible $\iff \underline{G} \Leftrightarrow \underline{G}_{wb}$ for a well-behaved 1-graph $\underline{G}_{wb}$ with $\text{depth} \leq (n+1)^3 \cdot 2^{3k}$ and max. out-degree $\leq k$.

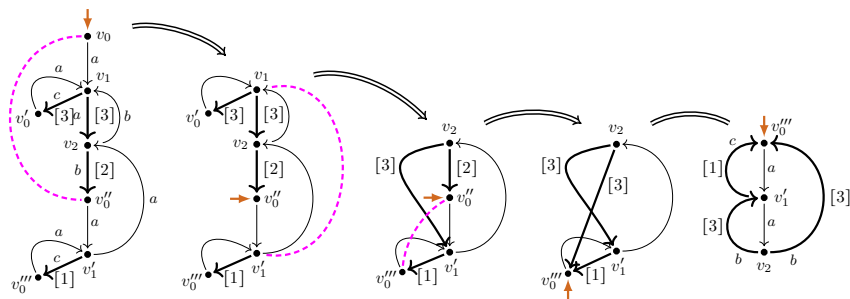
Theorem (*Baeten–Corradini–G, 2005*)

$\llbracket \cdot \rrbracket_P$ -Expressibility of graphs is *decidable* (at least doubly-exponentially).

Small steps for hard questions

- ▶ Regular expressions (also: 1-free/under-star-1-free $(*/\perp)$ expr.s)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility questions)
 - ▶ axioms for $\llbracket \cdot \rrbracket_P$ -identity ($\leadsto$ completeness question)
- ▶ **Small steps** for **hard questions**
 - (st1) reduction steps for well-behaved graphs under bisimilarity
 - ▶ $\llbracket \cdot \rrbracket_P$ -expressibility is decidable
 - (st2) collapse steps for **LLEE** graphs under bisimilarity
 - ▶ completeness for $(*/\perp)$ expressions
 - (st3) collapse steps for **LLEE** 1-graphs under bisimilarity, as far as possible
 - ▶ counterexample to collapse, but leads to completeness
- ▶ Interpretation–readback correspondences
 - (irc) **LLEE**/**1-LLEE** graphs $\hat{=}$ image of $P^\bullet$ restricted to $(*/\perp)$ /all expr.s
 - ▶ where $P^\bullet$ a compact refinement of P
 - ▶ crucial for P -/ $\llbracket \cdot \rrbracket_P$ -expressibility, and completeness questions
- ▶ Outlook

Collapse steps for LLEE graphs under bisimilarity



Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

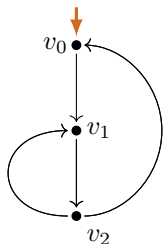
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to **it**.
- (L3) Immediate termination is **only** possible at the **start vertex**.

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.

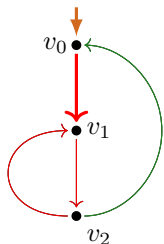


Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



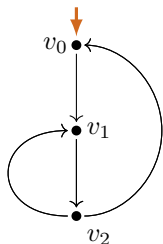
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



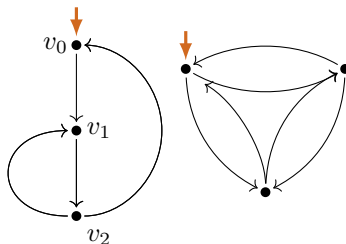
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



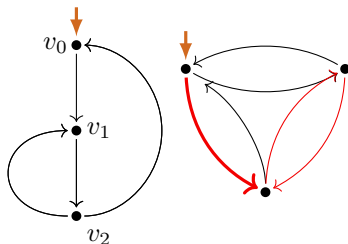
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



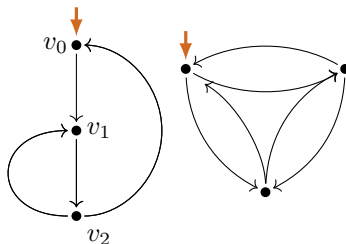
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



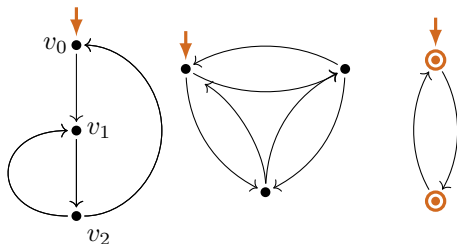
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



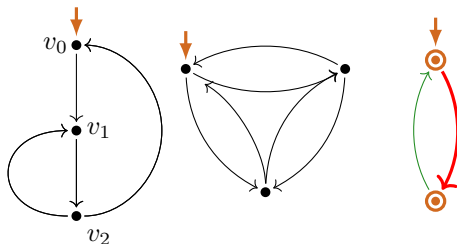
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



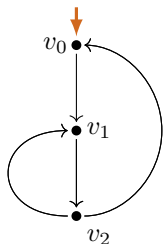
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

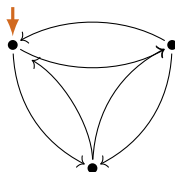
Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~

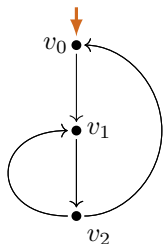


Loop graphs (interpretations of innermost iterations without 1)

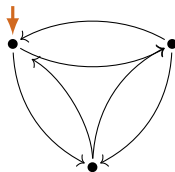
Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~

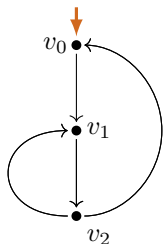


Loop graphs (interpretations of innermost iterations without 1)

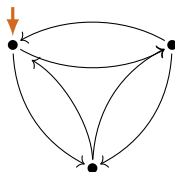
Definition

A process graph is a **loop graph** if:

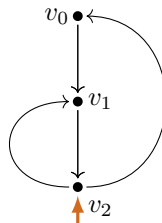
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~

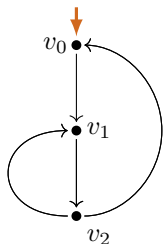


Loop graphs (interpretations of innermost iterations without 1)

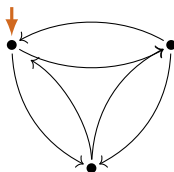
Definition

A process graph is a **loop graph** if:

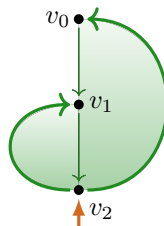
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~

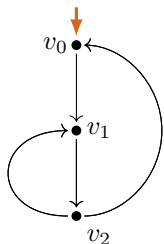


Loop graphs (interpretations of innermost iterations without 1)

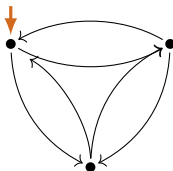
Definition

A process graph is a **loop graph** if:

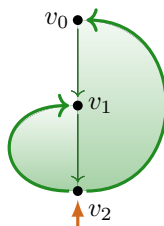
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



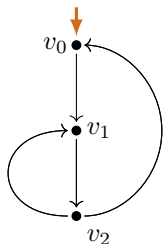
loop chart

Loop graphs (interpretations of innermost iterations without 1)

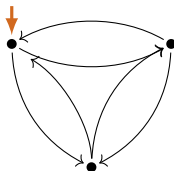
Definition

A process graph is a **loop graph** if:

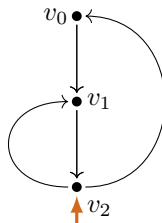
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



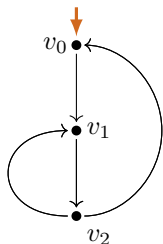
loop chart

Loop graphs (interpretations of innermost iterations without 1)

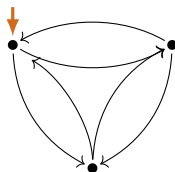
Definition

A process graph is a **loop graph** if:

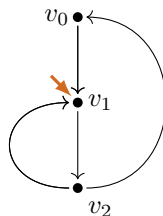
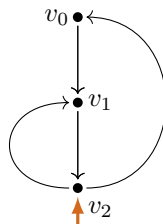
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~ **loop chart**

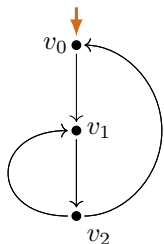


Loop graphs (interpretations of innermost iterations without 1)

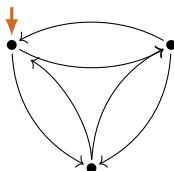
Definition

A process graph is a **loop graph** if:

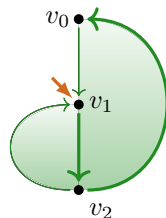
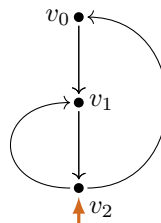
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~ **loop chart**

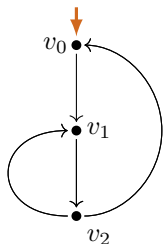


Loop graphs (interpretations of innermost iterations without 1)

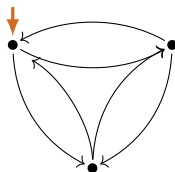
Definition

A process graph is a **loop graph** if:

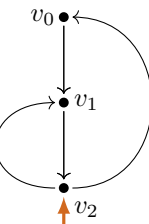
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



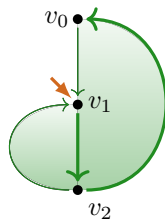
(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



loop chart



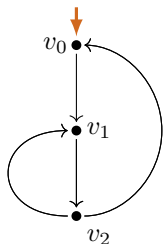
loop chart

Loop graphs (interpretations of innermost iterations without 1)

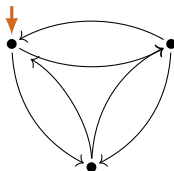
Definition

A process graph is a **loop graph** if:

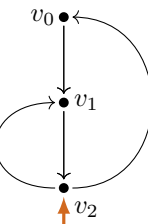
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



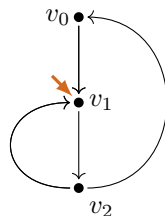
(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



loop chart



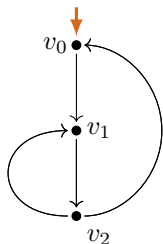
loop chart

Loop graphs (interpretations of innermost iterations without 1)

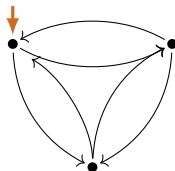
Definition

A process graph is a **loop graph** if:

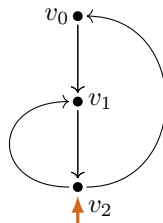
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



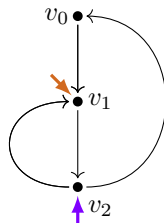
(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



loop chart

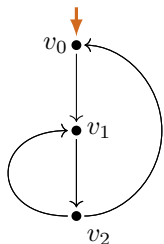


Loop graphs (interpretations of innermost iterations without 1)

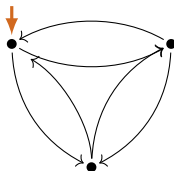
Definition

A process graph is a **loop graph** if:

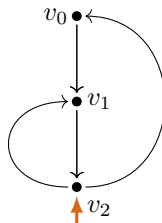
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



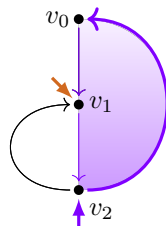
(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



loop chart

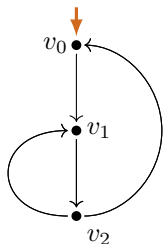


Loop graphs (interpretations of innermost iterations without 1)

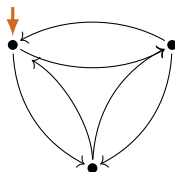
Definition

A process graph is a **loop graph** if:

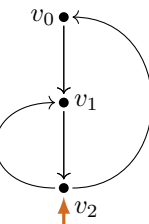
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



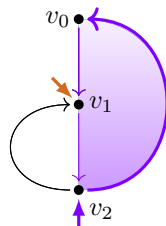
(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



loop chart



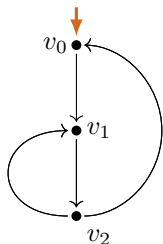
loop subchart

Loop graphs (interpretations of innermost iterations without 1)

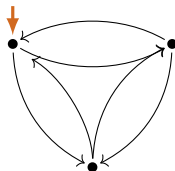
Definition

A process graph is a **loop graph** if:

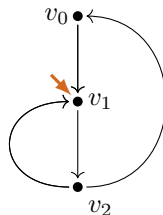
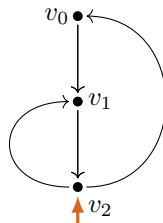
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~ **loop chart**



L

Definition

A graph G satisfies **L**EE (*loop existence and elimination*) if:

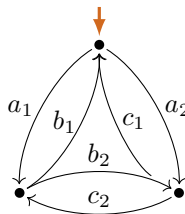
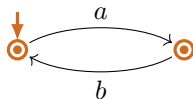
$$\exists G_0 \left(G \xrightarrow[\text{elim}]{*} G_0 \not\rightarrow_{\text{elim}} \wedge G_0 \text{ permits no infinite path} \right).$$

L

Definition

A graph G satisfies **L**EE (*loop existence and elimination*) if:

$$\exists G_0 \left(G \xrightarrow{*}_{\text{elim}} G_0 \not\rightarrow_{\text{elim}} \right. \\ \left. \wedge G_0 \text{ permits no infinite path} \right).$$

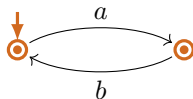


L

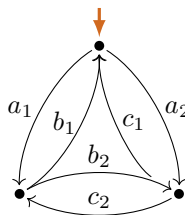
Definition

A graph G satisfies **L**EE (*loop existence and elimination*) if:

$$\exists G_0 \left(G \xrightarrow{*}_{\text{elim}} G_0 \not\rightarrow_{\text{elim}} \right. \\ \left. \wedge G_0 \text{ permits no infinite path} \right).$$

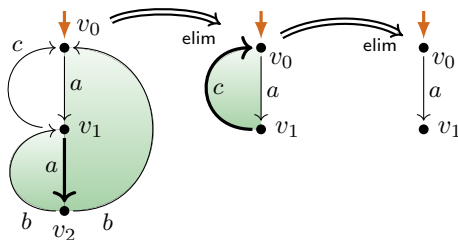


not LEE

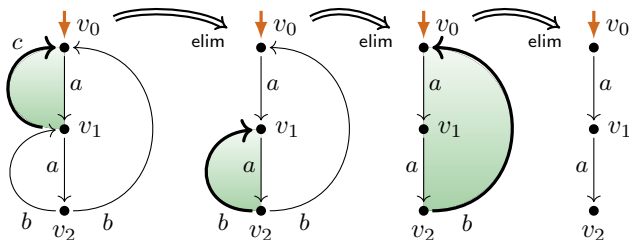
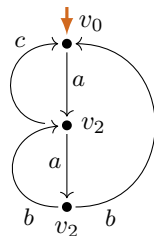


not LEE

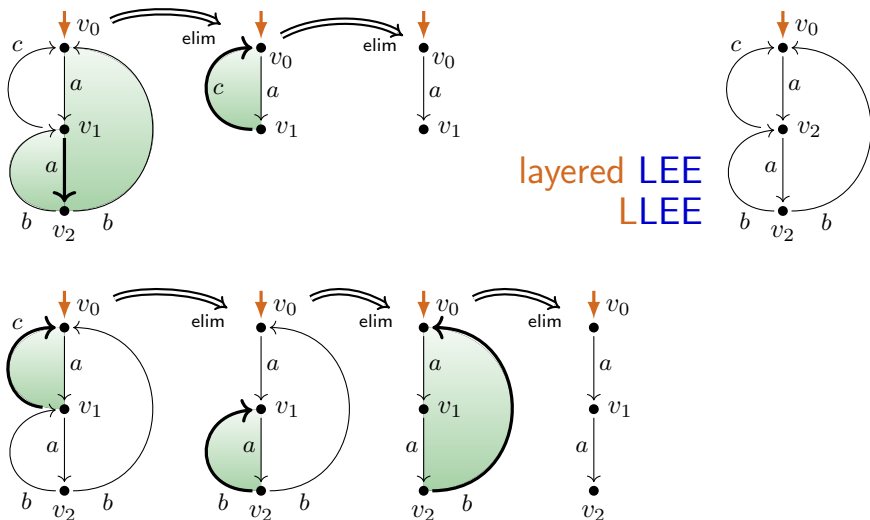
LEE



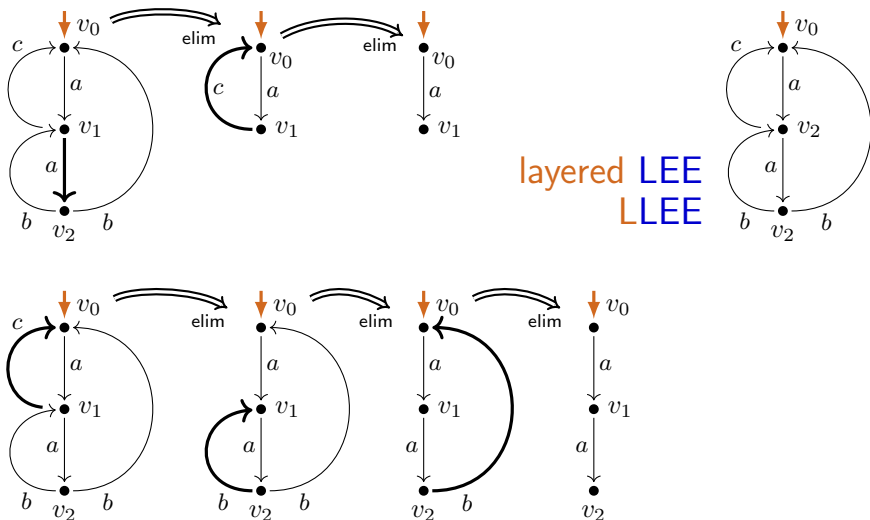
LEE



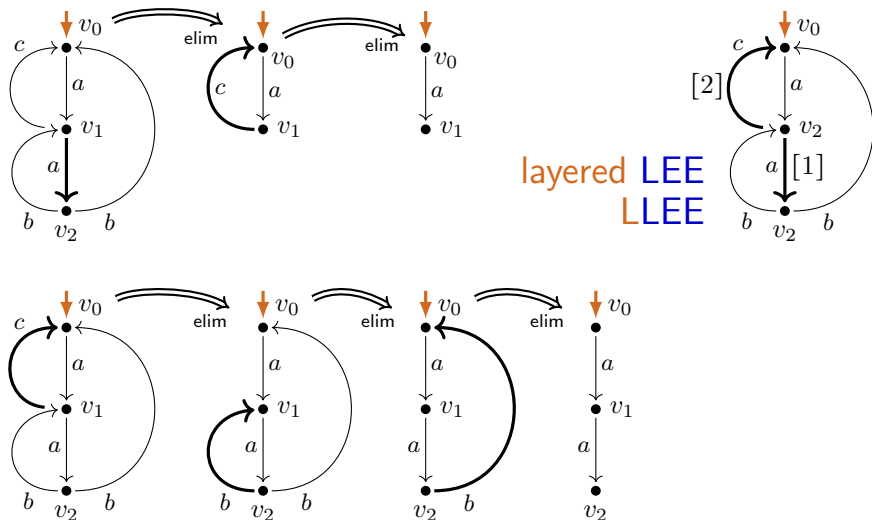
Layered LEE



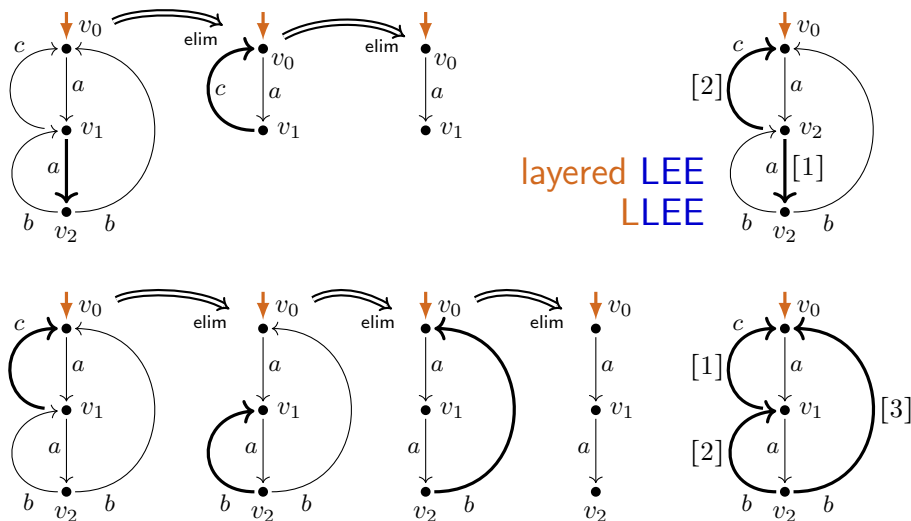
Layered LEE



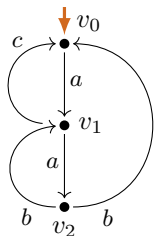
Layered LEE



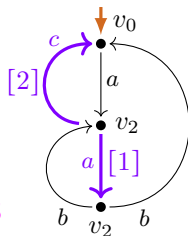
LLEE-witness and LLEE-graph



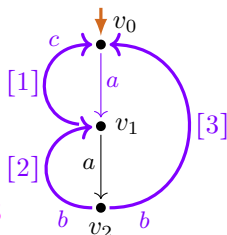
LLEE-witness and LLEE-graph



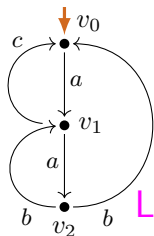
LLEE-witness



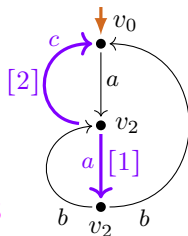
LLEE-witness



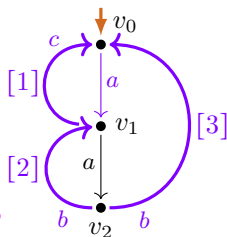
LLEE-witness and LLEE-graph



LLEE-graph

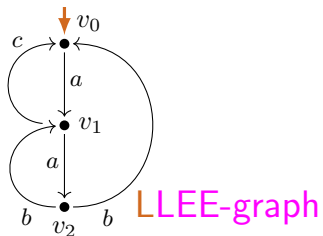


LLEE-witness

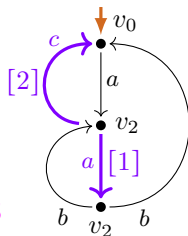


LLEE-witness

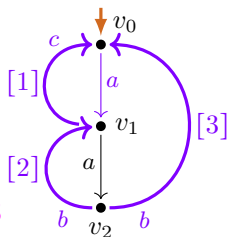
LLEE-witness and LLEE-graph



layered
LLEE-witness



layered
LLEE-witness



Properties of LEE-graphs

Theorem ($\Leftarrow$ *G-Fokkink, 2020*)

A process graph G

is $\llbracket \cdot \rrbracket_P$ -expressible by an $(*/\perp)$ regular expression

(i.e. P -expressible modulo bisimilarity by an $(*/\perp)$ reg. expr.)

if and only if

the bisimulation collapse of G satisfies LEE.

Properties of LEE-graphs

Theorem ($\Leftarrow$ G-Fokkink, 2020)

A process graph G

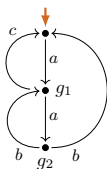
is $\llbracket \cdot \rrbracket_P$ -expressible by an $(*/\pm)$ regular expression

(i.e. P -expressible modulo bisimilarity by an $(*/\pm)$ reg. expr.)

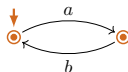
if and only if

the bisimulation collapse of G satisfies LEE.

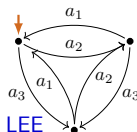
Hence $\llbracket \cdot \rrbracket_P$ -expressible | **not** $\llbracket \cdot \rrbracket_P$ -expressible by $(*/\pm)$ regular expressions:



L EE



not L EE

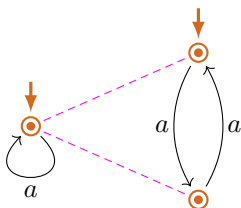


not L EE

L EE under bisimulation

Observation

- L EE is **not** invariant under bisimulation.



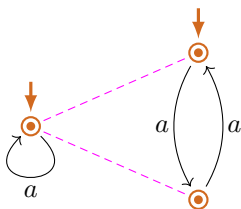
L EE

$\neg$ L EE

L EE under bisimulation

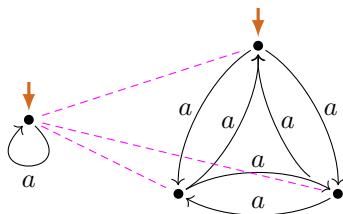
Observation

- L EE is **not** invariant under bisimulation.



LEE

¬LEE



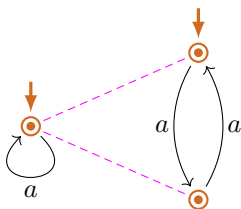
LEE

¬LEE

L EE under bisimulation

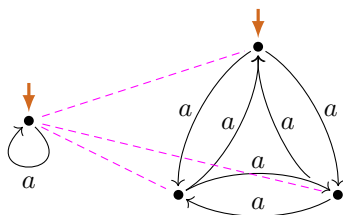
Observation

- ▶ L EE is **not** invariant under bisimulation.
- ▶ L EE is **not** preserved by converse functional bisimulation.



LEE

¬LEE



LEE

¬LEE

L_{EE} under functional bisimulation

Lemma

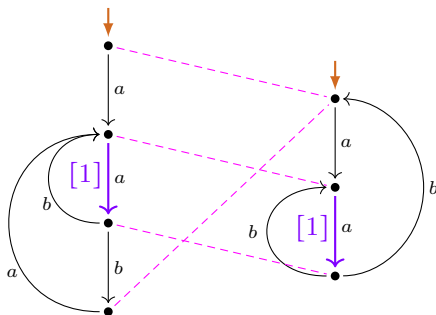
(i) L_{EE} is preserved by *functional bisimulations*:

$$\text{LEE}(G_1) \wedge G_1 \rhd G_2 \implies \text{LEE}(G_2).$$

Proof (Idea).

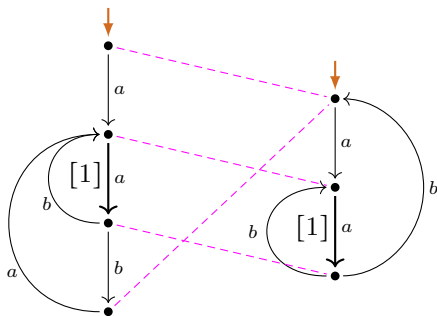
Use loop elimination in G_1 to carry out loop elimination in G_2 .

Collapsing LLEE-witnesses: global argument

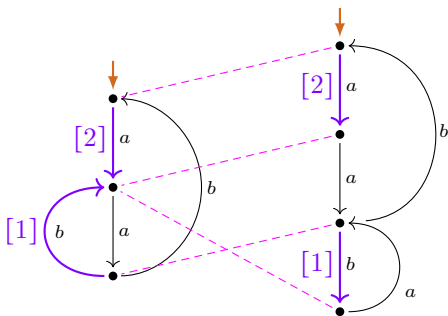


$$P(a(a(b + ba))^* \cdot 0)$$

Collapsing LLEE-witnesses: global argument



$$P(a(a(b+ba))^* \cdot 0)$$



$$P((aa(ba))^* \cdot b)^* \cdot 0$$

L_{EE} under functional bisimulation / bisimulation collapse

Lemma

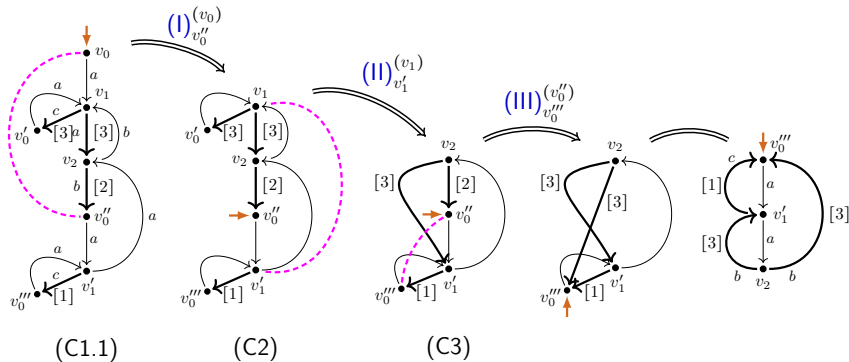
(i) L_{EE} is preserved by *functional bisimulations*:

$$\text{LEE}(G_1) \wedge G_1 \rightrightarrows G_2 \implies \text{LEE}(G_2).$$

(ii) L_{EE} is preserved from a process graph to its *bisimulation collapse*:

$$\text{LEE}(G) \wedge G \text{ has bisimulation collapse } C \implies \text{LEE}(C).$$

LLEE-preserving collapse of LLEE-graphs: stepwise proof (no 1-transitions!)

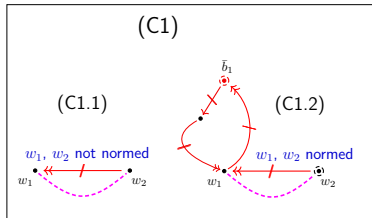


Lemma

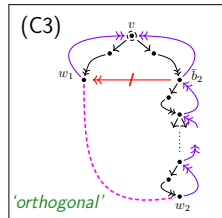
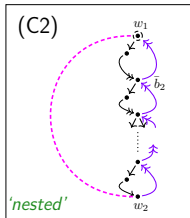
The bisimulation collapse of a LLEE-graph is again a LLEE-graph.

Reduced bisimilarity redundancies in LLEE-graphs (no 1-trans.!) (G-Fokkink, LICS'20)

w_1, w_2 in different scc's



w_1, w_2 in the same scc

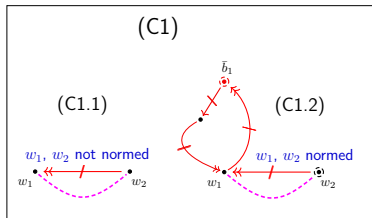


Lemma

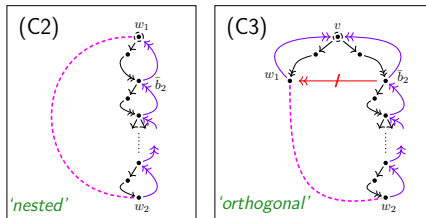
Every *not collapsed* LLEE-graph contains bisimilar vertices $w_1 \neq w_2$ of kind (C1), (C2), or (C3) (a *reduced bisimilarity redundancy* $\langle w_1, w_2 \rangle$):

Reduced bisimilarity redundancies in LLEE-graphs (no 1-trans.!) (G-Fokkink, LICS'20)

w_1, w_2 in different scc's



w_1, w_2 in the same scc



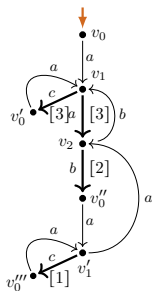
Lemma

Every *not collapsed* LLEE-graph contains bisimilar vertices $w_1 \neq w_2$ of kind (C1), (C2), or (C3) (a *reduced bisimilarity redundancy* $\langle w_1, w_2 \rangle$):

Lemma

Every *reduced bisimilarity redundancy* in a LLEE-graph can be eliminated LLEE-preservingly.

LLEE-preserving collapse of LLEE-graphs: stepwise proof (no 1-transitions!)

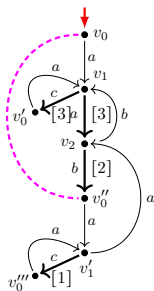


(C1.1)

Lemma

The bisimulation collapse of a LLEE-graph is again a LLEE-graph.

LLEE-preserving collapse of LLEE-graphs: stepwise proof (no 1-transitions!)

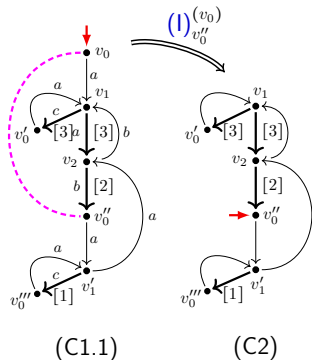


(C1.1)

Lemma

The bisimulation collapse of a LLEE-graph is again a LLEE-graph.

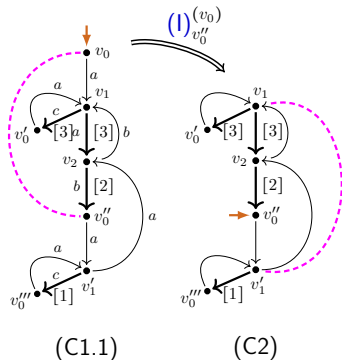
LLEE-preserving collapse of LLEE-graphs: stepwise proof (no 1-transitions!)



Lemma

The bisimulation collapse of a LLEE-graph is again a LLEE-graph.

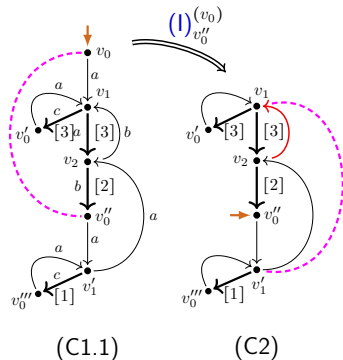
LLEE-preserving collapse of LLEE-graphs: stepwise proof (no 1-transitions!)



Lemma

The bisimulation collapse of a LLEE-graph is again a LLEE-graph.

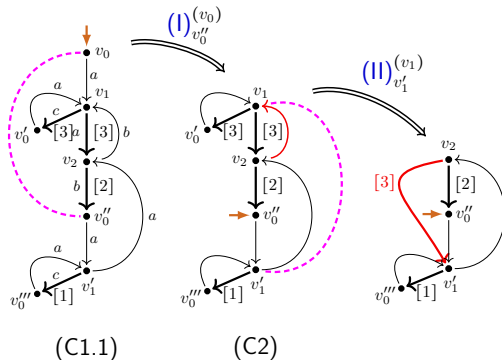
LLEE-preserving collapse of LLEE-graphs: stepwise proof (no 1-transitions!)



Lemma

The bisimulation collapse of a LLEE-graph is again a LLEE-graph.

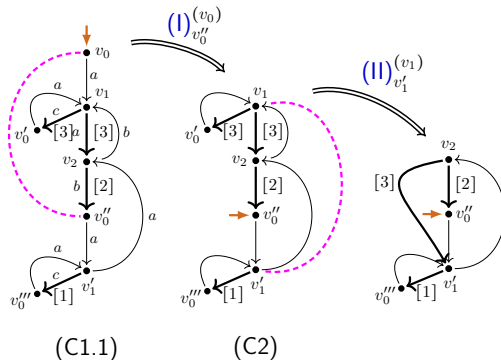
LLEE-preserving collapse of LLEE-graphs: stepwise proof (no 1-transitions!)



Lemma

The bisimulation collapse of a LLEE-graph is again a LLEE-graph.

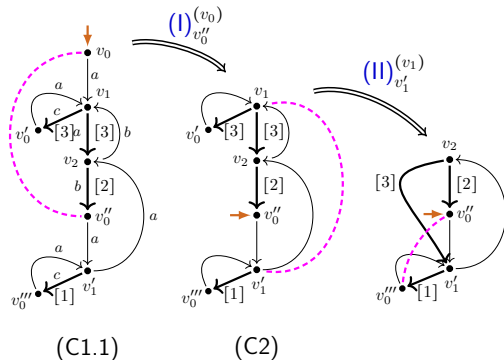
LLEE-preserving collapse of LLEE-graphs: stepwise proof (no 1-transitions!)



Lemma

The bisimulation collapse of a LLEE-graph is again a LLEE-graph.

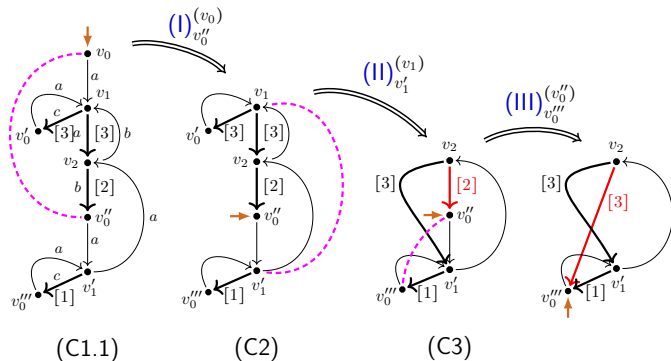
LLEE-preserving collapse of LLEE-graphs: stepwise proof (no 1-transitions!)



Lemma

The bisimulation collapse of a LLEE-graph is again a LLEE-graph.

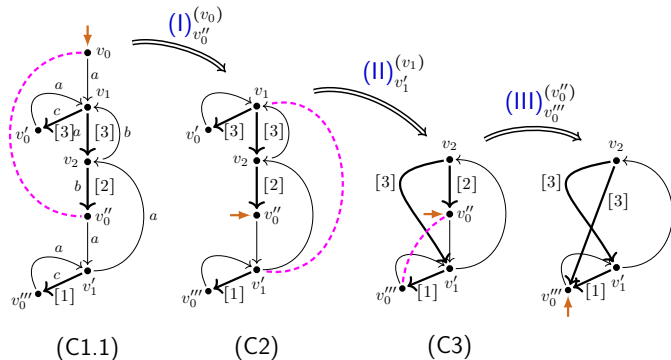
LLEE-preserving collapse of LLEE-graphs: stepwise proof (no 1-transitions!)



Lemma

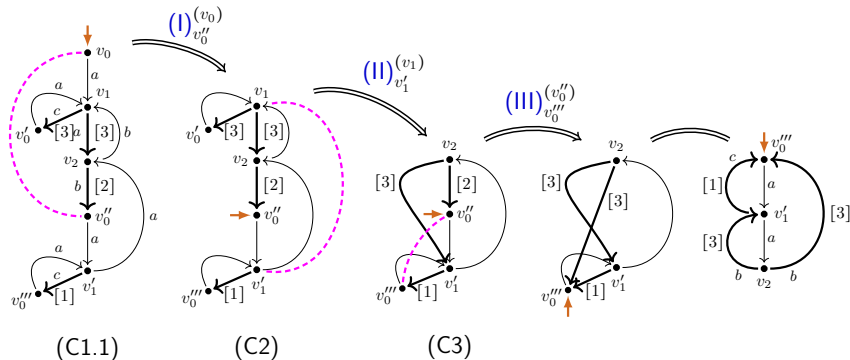
The bisimulation collapse of a LLEE-graph is again a LLEE-graph.

LLEE-preserving collapse of LLEE-graphs: stepwise proof (no 1-transitions!)



Lemma

The bisimulation collapse of a LLEE-graph is again a LLEE-graph.



Lemma

The bisimulation collapse of a LLEE-graph is again a LLEE-graph.

Results facilitated by steps (st2)

Corollary ($\Leftarrow$ *G-Fokkink, LICS 2020*)

For every process graph G , TFAE:

- (i) G is $\llbracket \cdot \rrbracket_P$ -expressible by a $(*/\perp)$ regular expression
- (ii) The bisimulation collapse of G satisfies LLEE.
- (iii) The bisim. collapse of G is $\llbracket \cdot \rrbracket_P$ -expressible by a $(*/\perp)$ reg. expression.

Results facilitated by steps (st2)

Corollary ($\Leftarrow$ G-Fokkink, LICS 2020)

For every process graph G , TFAE:

- (i) G is $\llbracket \cdot \rrbracket_P$ -expressible by a $(*/\pm)$ regular expression
- (ii) The bisimulation collapse of G satisfies LLEE.
- (iii) The bisim. collapse of G is $\llbracket \cdot \rrbracket_P$ -expressible by a $(*/\pm)$ reg. expression.

Lemma (IWC 2026)

LLEE, LLEE $\in$ PTIME ($\leq$ cubic in graph size).

Corollary

$\llbracket \cdot \rrbracket_P$ -expressibility by a $(*/\pm)$ regular expression $\in$ PTIME.

Results facilitated by steps (st2)

Corollary ($\Leftarrow$ *G-Fokkink, LICS 2020*)

For every process graph G , TFAE:

- (i) G is $\llbracket \cdot \rrbracket_P$ -expressible by a $(*/\pm)$ regular expression
- (ii) The bisimulation collapse of G satisfies LLEE.
- (iii) The bisim. collapse of G is $\llbracket \cdot \rrbracket_P$ -expressible by a $(*/\pm)$ reg. expression.

Lemma (*IWC 2026*)

LLEE, LLEE $\in$ PTIME ($\leq$ cubic in graph size).

Corollary

$\llbracket \cdot \rrbracket_P$ -expressibility by a $(*/\pm)$ regular expression $\in$ PTIME.

Theorem (*G-Fokkink, LICS 2020*)

Mil restricted to $(*/\pm)$ regular expressions is complete.

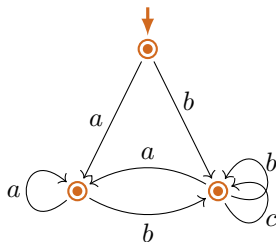
BBP (Bergstra, Bethke, Ponse)

Small steps for hard questions

- ▶ Regular expressions (also: 1-free/under-star-1-free $(*/\perp)$ expr.s)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility questions)
 - ▶ axioms for $\llbracket \cdot \rrbracket_P$ -identity ($\leadsto$ completeness question)
- ▶ **Small steps** for **hard questions**
 - (st1) reduction steps for well-behaved graphs under bisimilarity
 - ▶ $\llbracket \cdot \rrbracket_P$ -expressibility is decidable
 - (st2) collapse steps for LLEE graphs under bisimilarity
 - ▶ completeness for $(*/\perp)$ expressions
 - (st3) collapse steps for LLEE 1-graphs under bisimilarity, **as far as possible**
 - ▶ counterexample to collapse, but leads to completeness
- ▶ Interpretation–readback correspondences
 - (irc) LLEE/1-LLEE graphs $\hat{=}$ image of $P^\bullet$ restricted to $(*/\perp)$ /all expr.s
 - ▶ where $P^\bullet$ a compact refinement of P
 - ▶ crucial for P -/ $\llbracket \cdot \rrbracket_P$ -expressibility, and completeness questions
- ▶ Outlook

1-Graphs with LLEE $\hat{=}$ graphs with 1-LLEE

graph

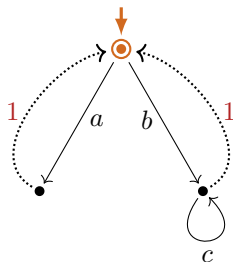


not LLEE

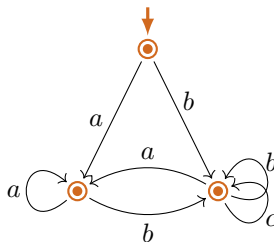
$$P((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

1-Graphs with LLEE $\hat{=}$ graphs with 1-LLEE

1-graph



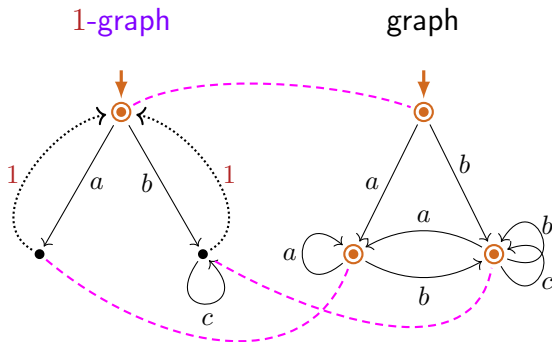
graph



$$\underline{P}((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

$$P((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

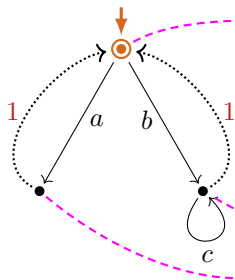
1-Graphs with LLEE $\hat{=}$ graphs with 1-LLEE



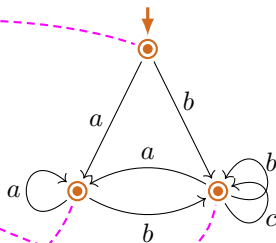
$$\underline{P}((a \cdot 1 + b \cdot (c^* \cdot 1))^*) \quad P((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

1-Graphs with LLEE $\hat{=}$ graphs with 1-LLEE

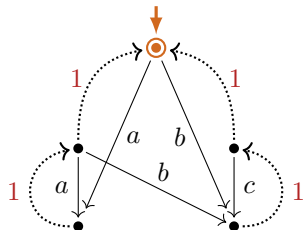
1-graph



graph



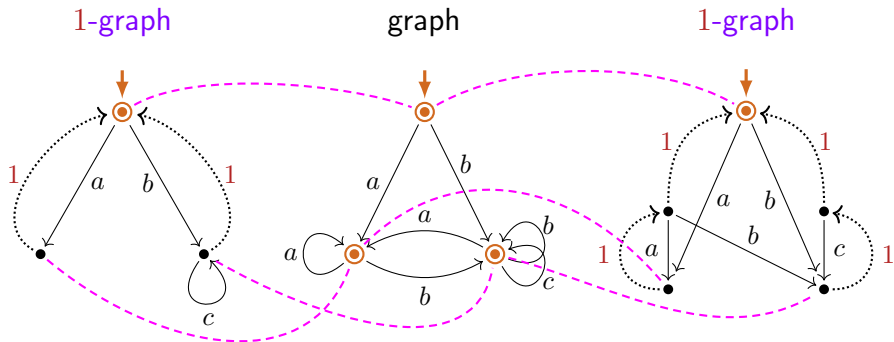
1-graph



$$\underline{P}((a \cdot 1 + b \cdot (c^* \cdot 1))^*) \quad \underline{P}((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

$$\underline{P}((a \cdot (1 \cdot ((a \cdot 1)^* \cdot 1)) \cdot b \cdot ((1 \cdot (c \cdot 1)^*)) \cdot 1 + b \cdot ((1 \cdot (c \cdot 1)^*)) \cdot 1)^*)$$

1-Graphs with LLEE $\hat{=}$ graphs with 1-LLEE



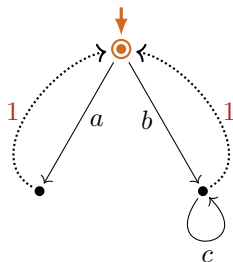
$$\underline{P}((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

$$\underline{P}((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

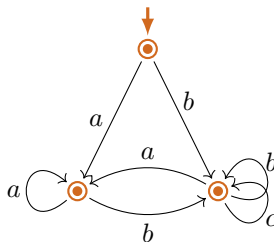
$$\underline{P}((a \cdot (1 \cdot ((a \cdot 1)^* \cdot 1)) \cdot b \cdot ((1 \cdot (c \cdot 1)^*)) \cdot 1 + b \cdot ((1 \cdot (c \cdot 1)^*)) \cdot 1)^*)$$

1-Graphs with LLEE $\hat{=}$ graphs with 1-LLEE

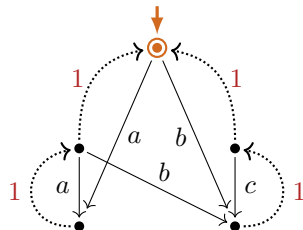
1-graph



graph



1-graph



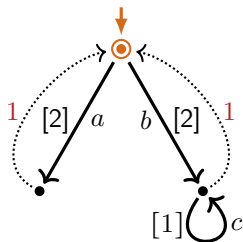
$$\underline{P}((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

$$P((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

$$\underline{P}((a \cdot (1 \cdot ((a \cdot 1)^* \cdot 1)) \cdot b \cdot ((1 \cdot (c \cdot 1)^*)) \cdot 1 + b \cdot ((1 \cdot (c \cdot 1)^*)) \cdot 1)^*)$$

1-Graphs with LLEE $\hat{=}$ graphs with 1-LLEE

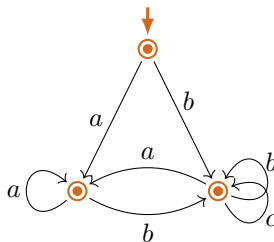
1-graph



LLEE

$$\underline{P}((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

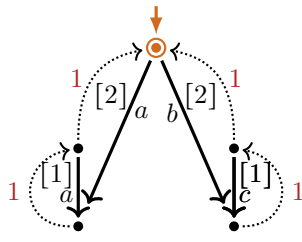
graph



not LLEE

$$\underline{P}((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

1-graph

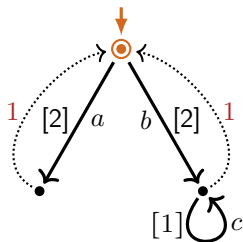


LLEE

$$\underline{P}((a \cdot (1 \cdot ((a \cdot 1)^* \cdot 1)) \cdot b \cdot ((1 \cdot (c \cdot 1)^*)) \cdot 1 + b \cdot ((1 \cdot (c \cdot 1)^*)) \cdot 1)^*)$$

1-Graphs with LLEE $\hat{=}$ graphs with 1-LLEE

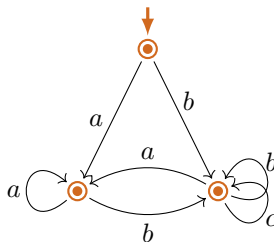
1-graph



LLEE

$$\underline{P}((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

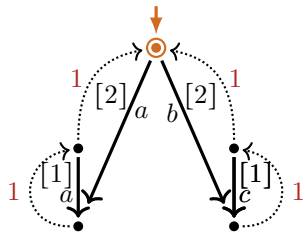
graph



1-LLEE

$$\underline{P}((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$

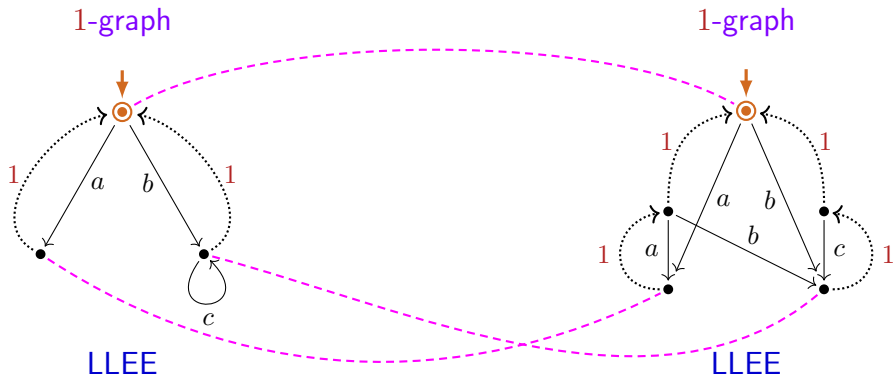
1-graph



LLEE

$$\underline{P}((a \cdot (1 \cdot ((a \cdot 1)^* \cdot 1)) \cdot b \cdot ((1 \cdot (c \cdot 1)^*)) \cdot 1 + b \cdot ((1 \cdot (c \cdot 1)^*)) \cdot 1)^*)$$

1-Graphs with LLEE $\hat{=}$ graphs with 1-LLEE

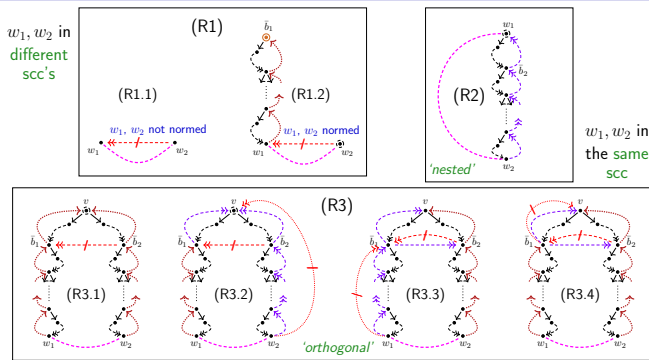


$$\underline{P}((a \cdot 1 + b \cdot (c^* \cdot 1))^*)$$



$$\underline{P}((a \cdot (1 \cdot ((a \cdot 1)^* \cdot 1)) \cdot b \cdot ((1 \cdot (c \cdot 1)^*)) \cdot 1 + b \cdot ((1 \cdot (c \cdot 1)^*)) \cdot 1)^*)$$

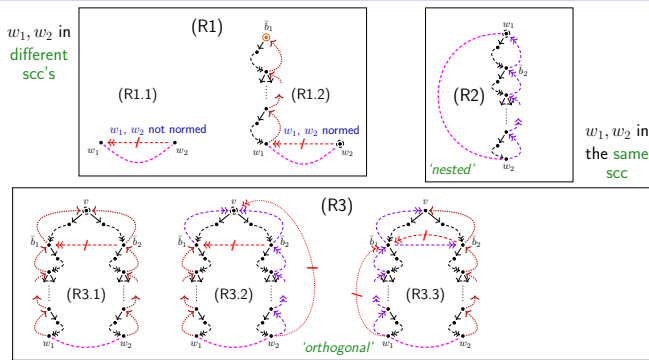
Reduced 1-bisimilarity redundancies in LLEE-1-graphs



Lemma

Every *not collapsed* LLEE-1-graph contains 1-bisimilar vertices $w_1 \neq w_2$ (a *reduced 1-bisimilarity redundancy* $\langle w_1, w_2 \rangle$) of kind (R1), (R2), (R3).

Reduced 1-bisimilarity redundancies in LLEE-1-graphs



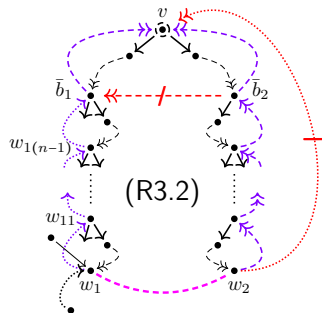
Lemma

Every *not collapsed* LLEE-1-graph contains 1-bisimilar vertices $w_1 \neq w_2$ (a *reduced 1-bisimilarity redundancy* $\langle w_1, w_2 \rangle$) of kind (R1), (R2), (R3).

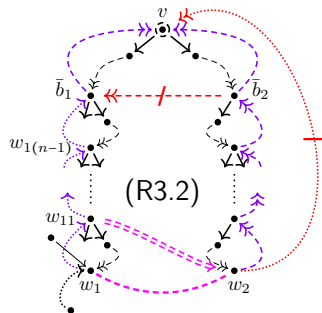
Lemma

Every *simple* reduced 1-bisimilarity redundancies in a LLEE-1-graph can be eliminated LLEE-preservingly.

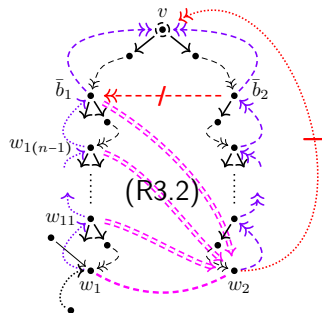
Elimination step of bisimilarity redundancies (R3.2)



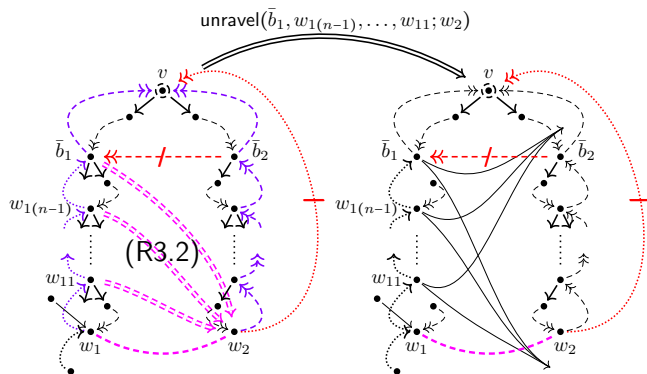
Elimination step of bisimilarity redundancies (R3.2)



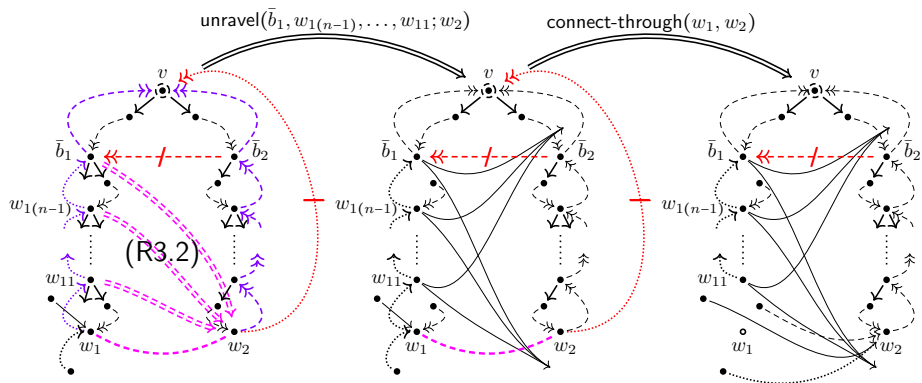
Elimination step of bisimilarity redundancies (R3.2)



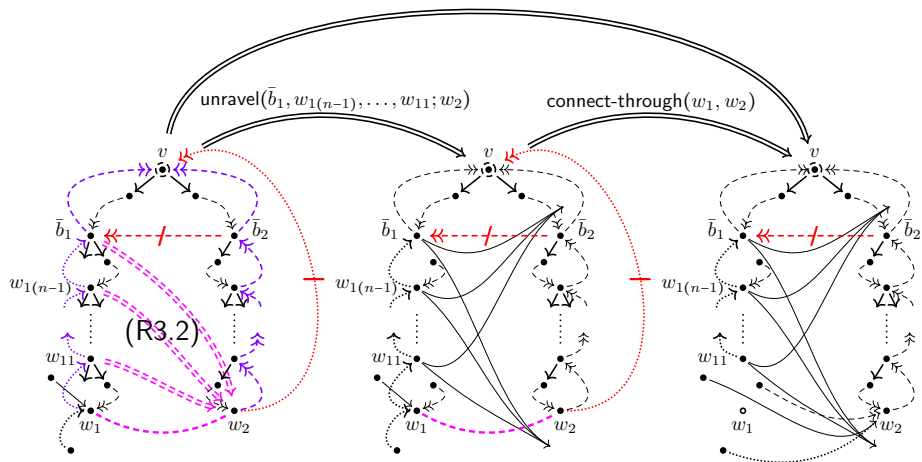
Elimination step of bisimilarity redundancies (R3.2)



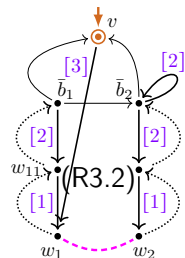
Elimination step of bisimilarity redundancies (R3.2)



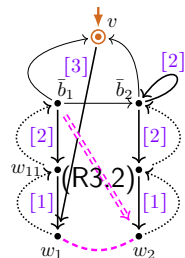
Elimination step of bisimilarity redundancies (R3.2)



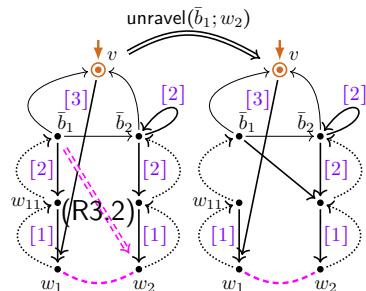
Elimination step of bisim. redundancy (R3.2) (example)



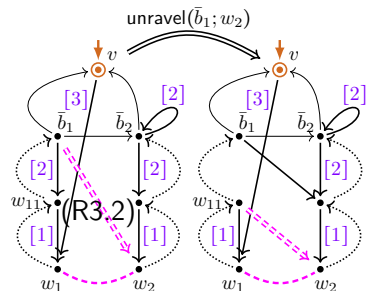
Elimination step of bisim. redundancy (R3.2) (example)



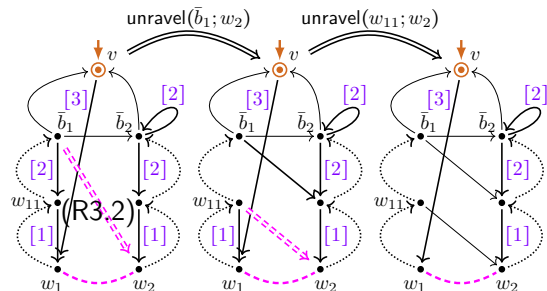
Elimination step of bisim. redundancy (R3.2) (example)



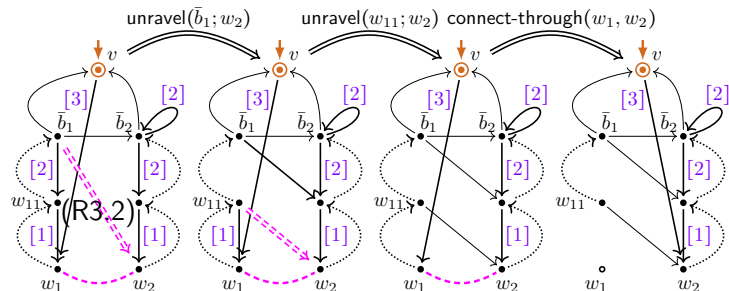
Elimination step of bisim. redundancy (R3.2) (example)



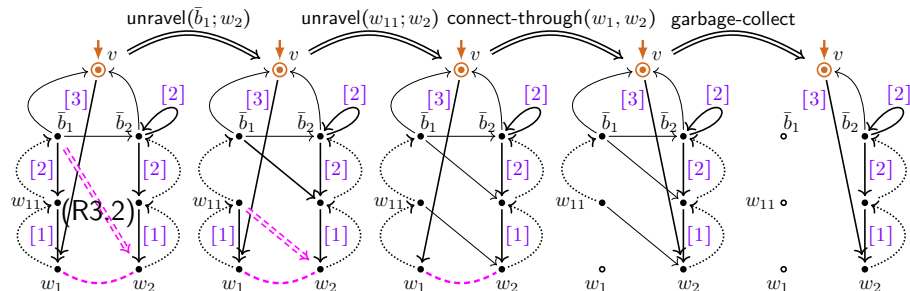
Elimination step of bisim. redundancy (R3.2) (example)



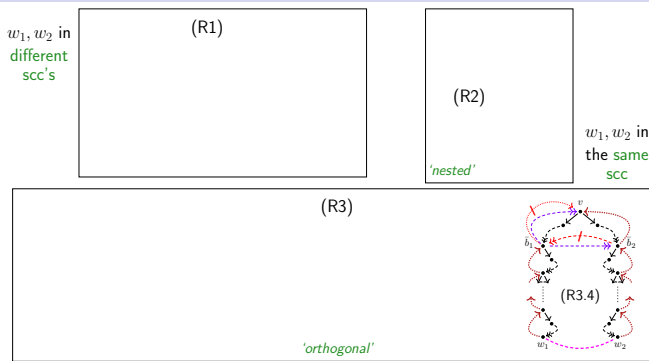
Elimination step of bisim. redundancy (R3.2) (example)



Elimination step of bisim. redundancy (R3.2) (example)



Reduced 1-bisimilarity redundancies in LLEE-1-graphs



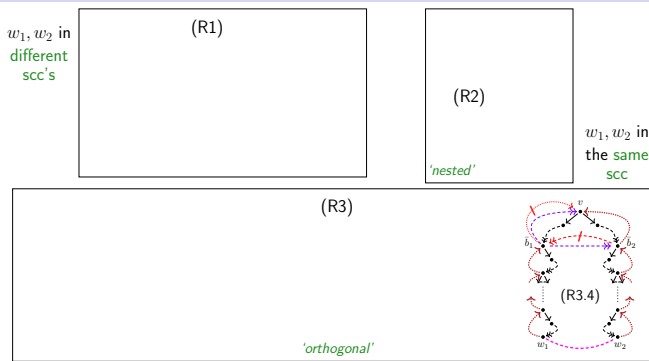
Lemma

Every *not collapsed* LLEE-1-graph contains 1-bisimilar vertices $w_1 \neq w_2$ (a *reduced 1-bisimilarity redundancy* $\langle w_1, w_2 \rangle$) of kind (R1), (R2), (R3).

Stumbling Block

How to LLEE-preservingly eliminate reduced 1-bisimilarity redundancies of kind (R3.4)?

Reduced 1-bisimilarity redundancies in LLEE-1-graphs



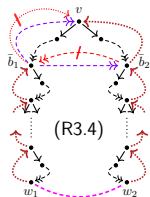
Lemma

Every *not collapsed* LLEE-1-graph contains 1-bisimilar vertices $w_1 \neq w_2$ (a *reduced 1-bisimilarity redundancy* $\langle w_1, w_2 \rangle$) of kind (R1), (R2), (R3).

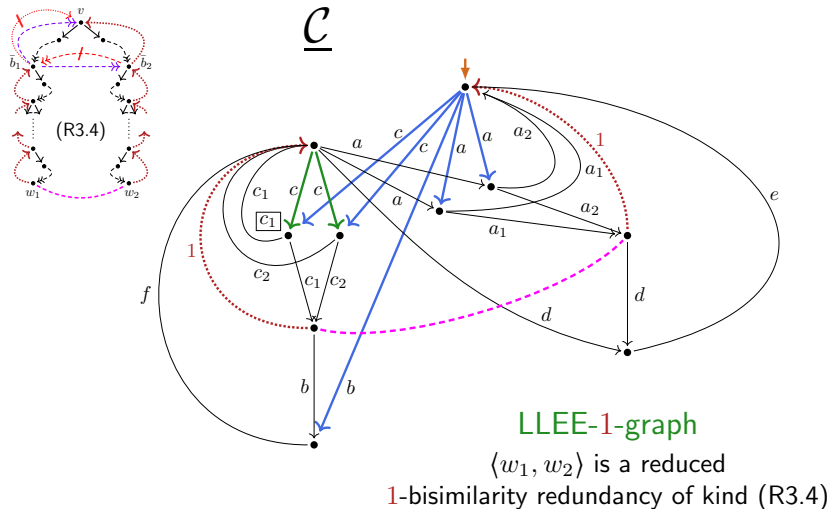
Stumbling Block

How to LLEE-preservingly eliminate
precrySTALLine reduced 1-bisimilarity redundancies?

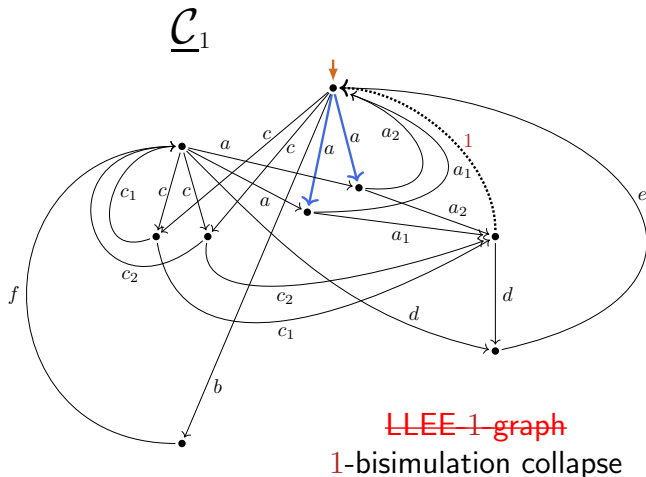
Counterexample LLEE-preserving collapse



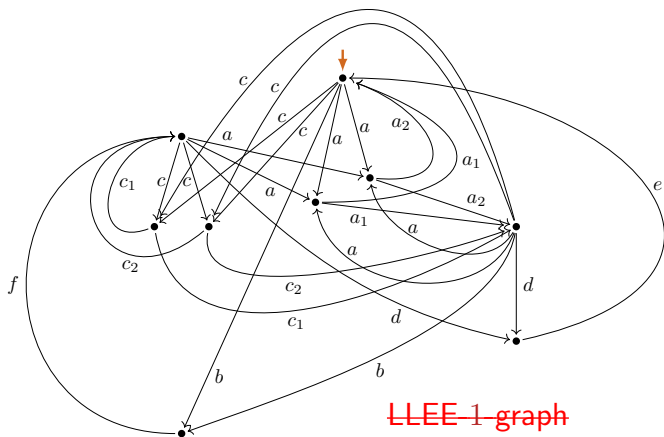
Counterexample LLEE-preserving collapse



Counterexample LLEE-preserving collapse

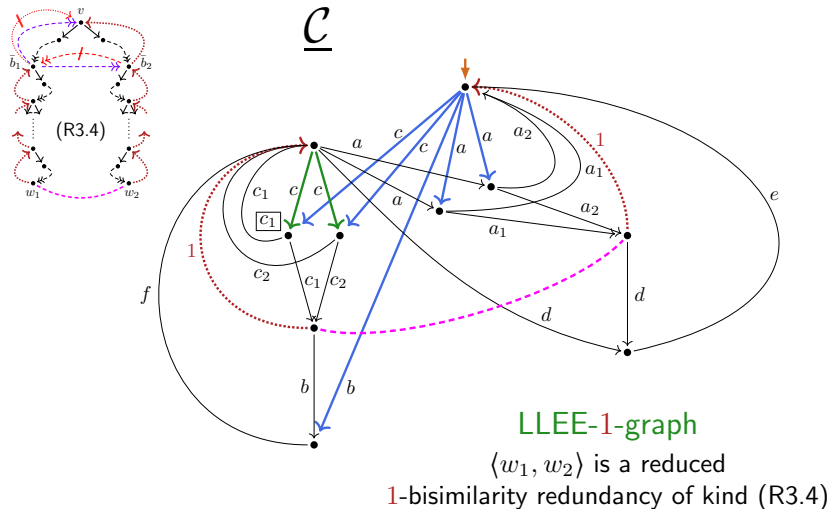


Counterexample LLEE-preserving collapse

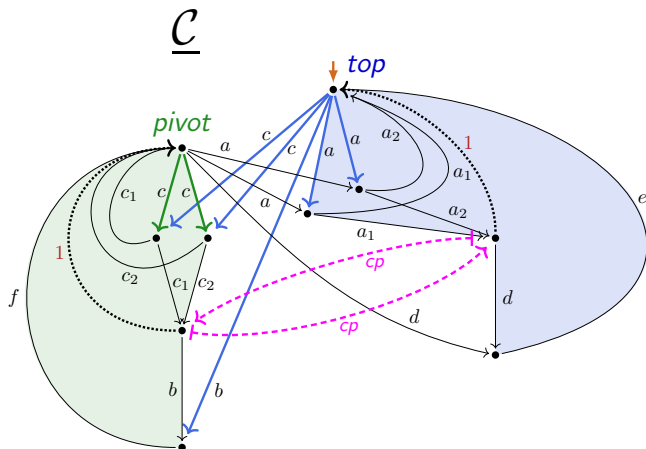


~~LLEE 1-graph~~
bisimulation collapse

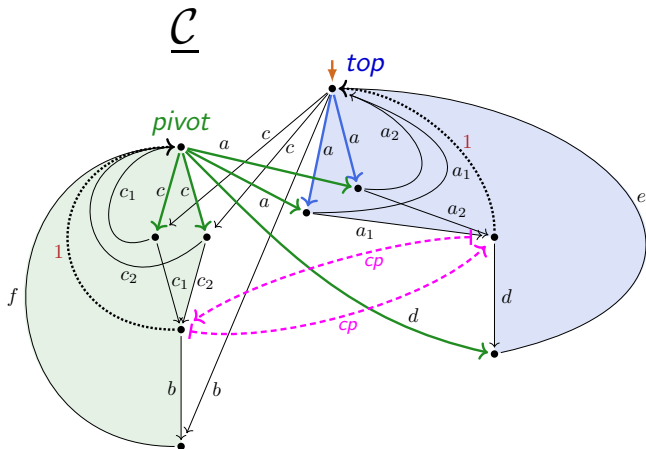
Counterexample LLEE-preserving collapse



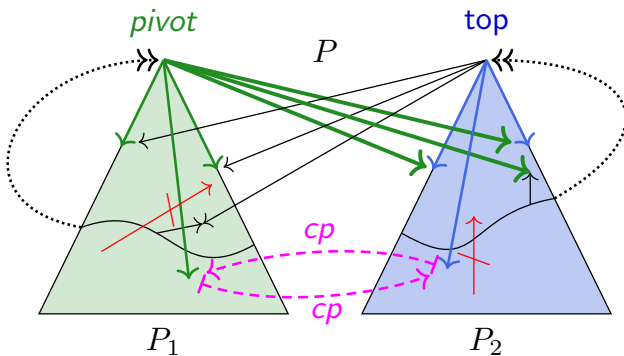
Twin-Crystal



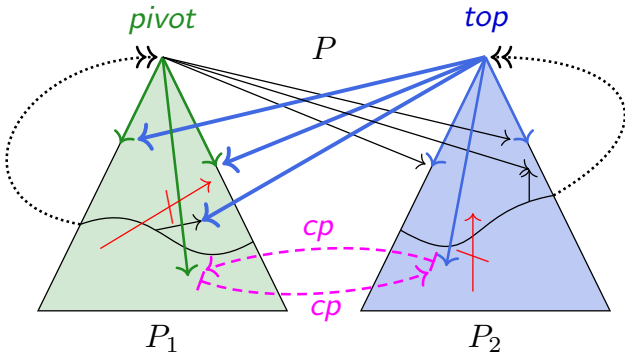
Twin-Crystal

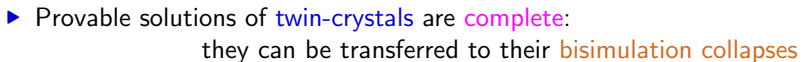


Twin-Crystal

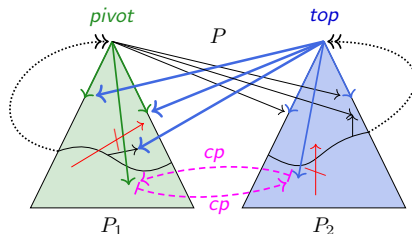


Twin-Crystal





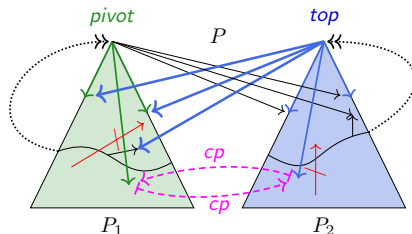
Crystallization



twin-crystal

Crystallized 1-graphs = LLEE-1-graphs that are collapsed
apart from some scc's that are twin-crystals.

Crystallization

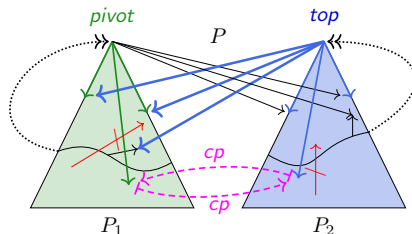


twin-crystal

Crystallized 1-graphs = LLEE-1-graphs that are collapsed apart from some scc's that are twin-crystals.

(CR) Crystallization: Every LLEE-1-graph can be reduced under bisimilarity to a bisimilar crystallized 1-graph.

Crystallization



twin-crystal

Crystallized 1-graphs = LLEE-1-graphs that are collapsed apart from some scc's that are twin-crystals.

(CR) Crystallization: Every LLEE-1-graph can be reduced under bisimilarity to a bisimilar crystallized 1-graph.

(CC) Every provable solution of a crystallized 1-graph gives rise to provable solution on the bisimulation collapse.

Results facilitated by steps (st3)

Proposition

The property 1-LLEE is not preserved under bisimulation collapse.

Results facilitated by steps (st3)

Proposition

The property 1-LLEE is not preserved under bisimulation collapse.

Crystallisation Lemma for LLEE-1-graphs

Every 1-graph $\underline{G}$ with LLEE can be pseudo-collapsed/crystallized by elimination steps for bisimilarity redundancies of kinds (R1), (R2), (R3.1), (R3.2), (R3.3) (and related steps) to a 1-graph $\underline{G}_0$ such that:

- ▶ $\underline{G}_0 \leftrightarrow \underline{G}$,
- ▶ $\underline{G}_0$ is crystallised: all strongly connected components are collapsed or twin-crystals.

Results facilitated by steps (st3)

Proposition

The property 1-LLEE is **not** preserved under bisimulation collapse.

Crystallisation Lemma for LLEE-1-graphs

Every 1-graph $\underline{G}$ with LLEE can be **pseudo-collapsed/crystallized** by **elimination steps** for bisimilarity redundancies of kinds (R1), (R2), (R3.1), (R3.2), (R3.3) (and related steps) to a 1-graph $\underline{G}_0$ such that:

- ▶ $\underline{G}_0 \leftrightarrow \underline{G}$,
- ▶ $\underline{G}_0$ is **crystallised**: all **strongly connected components** are collapsed or **twin-crystals**.

Crystallisation facilitates to **adapt** the **completeness proof** for **BBP** (that uses preservation of LLEE under collapse) via a **crystallisation detour**.

Results facilitated by steps (st3)

Proposition

The property 1-LLEE is **not** preserved under bisimulation collapse.

Crystallisation Lemma for LLEE-1-graphs

Every 1-graph $\underline{G}$ with LLEE can be **pseudo-collapsed/crystallized** by **elimination steps** for bisimilarity redundancies of kinds (R1), (R2), (R3.1), (R3.2), (R3.3) (and related steps) to a 1-graph $\underline{G}_0$ such that:

- ▶ $\underline{G}_0 \leftrightarrow \underline{G}$,
- ▶ $\underline{G}_0$ is **crystallised**: all **strongly connected components** are collapsed or **twin-crystals**.

Crystallisation facilitates to **adapt** the **completeness proof** for **BBP** (that uses preservation of LLEE under collapse) via a **crystallisation detour**.

Theorem (LICS 2022)

Milner's proof system **Mil** is **complete**.

Interpretation-readback correspondences

- ▶ Regular expressions (also: 1-free/under-star-1-free $(*/\perp)$ expr.s)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility questions)
 - ▶ axioms for $\llbracket \cdot \rrbracket_P$ -identity ($\leadsto$ completeness question)
- ▶ Small steps for hard questions
 - (st1) reduction steps for well-behaved graphs under bisimilarity
 - ▶ $\llbracket \cdot \rrbracket_P$ -expressibility is decidable
 - (st2) collapse steps for LLEE graphs under bisimilarity
 - ▶ completeness for $(*/\perp)$ expressions
 - (st3) collapse steps for LLEE 1-graphs under bisimilarity, as far as possible
 - ▶ counterexample to collapse, but leads to completeness
- ▶ Interpretation-readback correspondences
 - (irc) LLEE/1-LLEE graphs $\hat{=}$ image of $P^\bullet$ restricted to $(*/\perp)$ /all expr.s
 - ▶ where $P^\bullet$ a compact refinement of P
 - ▶ crucial for P -/ $\llbracket \cdot \rrbracket_P$ -expressibility, and completeness questions
- ▶ Outlook

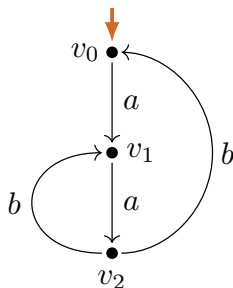
Readback

Lemma (*~ G-Fokkink, LICS 2020*)

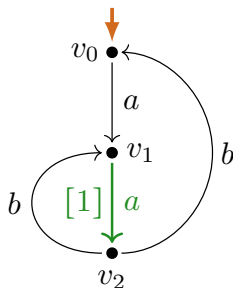
Process graphs G with LLEE are $\llbracket \cdot \rrbracket_P$ -expressible:

$$\text{LLEE}(G) \implies \exists e \in \text{RExp} \left(G \rightleftharpoons P(e) \right).$$

Readback from LLEE-witness (example)

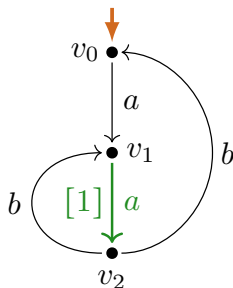


Readback from LLEE-witness (example)



layered
LEE-witness

Readback from LLEE-witness (example)



layered
LEE-witness

$$\begin{aligned} s(v_0) &= 0^* \cdot a \cdot s(v_1) \\ &=_{\text{Mil}^-} a \cdot s(v_1) \\ &=_{\text{Mil}^-} a \cdot (a \cdot (b + b \cdot a))^* \cdot 0 \end{aligned}$$

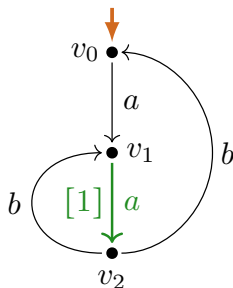
$$\begin{aligned} s(v_1) &= (a \cdot s(v_2, v_1))^* \cdot 0 \\ &=_{\text{Mil}^-} (a \cdot (b + b \cdot a))^* \cdot 0 \end{aligned}$$

$$\begin{aligned} s(v_2, v_1) &= 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1)) \\ &=_{\text{Mil}^-} 0^* \cdot (b \cdot 1 + b \cdot a) \\ &=_{\text{Mil}^-} b + b \cdot a \end{aligned}$$

$$\begin{aligned} s(v_1, v_1) &= 1 \\ s(v_0, v_1) &= 0^* \cdot a \cdot s(v_1, v_1) \\ &= 0^* \cdot a \cdot 1 \\ &=_{\text{Mil}^-} a \end{aligned}$$

Readback from layered LEE-witness (example)

$$s(v_0) = 0^* \cdot a \cdot s(v_1)$$

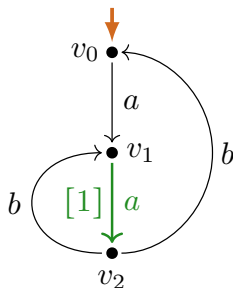


layered
LEE-witness

Readback from layered LEE-witness (example)

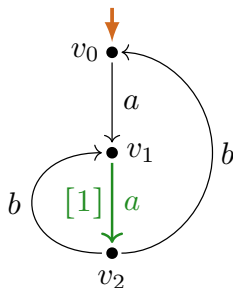
$$s(v_0) = 0^* \cdot a \cdot s(v_1)$$

$$s(v_1) = (a \cdot s(v_2, v_1))^* \cdot 0$$



layered
LEE-witness

Readback from layered LEE-witness (example)



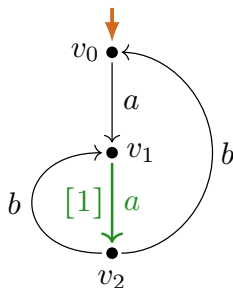
layered
LEE-witness

$$s(v_0) = 0^* \cdot a \cdot s(v_1)$$

$$s(v_1) = (a \cdot s(v_2, v_1))^* \cdot 0$$

$$s(v_2, v_1) = 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1))$$

Readback from layered LEE-witness (example)



layered
LEE-witness

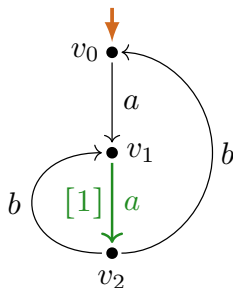
$$s(v_0) = 0^* \cdot a \cdot s(v_1)$$

$$s(v_1) = (a \cdot s(v_2, v_1))^* \cdot 0$$

$$s(v_2, v_1) = 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1))$$

$$s(v_1, v_1) = 1$$

Readback from layered LEE-witness (example)



layered
LEE-witness

$$s(v_0) = 0^* \cdot a \cdot s(v_1)$$

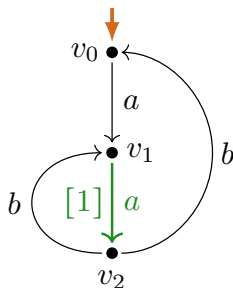
$$s(v_1) = (a \cdot s(v_2, v_1))^* \cdot 0$$

$$s(v_2, v_1) = 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1))$$

$$s(v_1, v_1) = 1$$

$$s(v_0, v_1) = 0^* \cdot a \cdot s(v_1, v_1)$$

Readback from layered LEE-witness (example)



layered
LEE-witness

$$s(v_0) = 0^* \cdot a \cdot s(v_1)$$

$$s(v_1) = (a \cdot s(v_2, v_1))^* \cdot 0$$

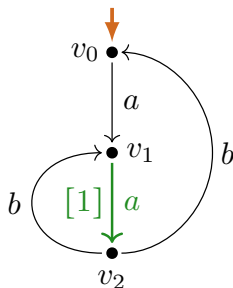
$$s(v_2, v_1) = 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1))$$

$$s(v_1, v_1) = 1$$

$$s(v_0, v_1) = 0^* \cdot a \cdot s(v_1, v_1)$$

$$= 0^* \cdot a \cdot 1$$

Readback from layered LEE-witness (example)



layered
LEE-witness

$$s(v_0) = 0^* \cdot a \cdot s(v_1)$$

$$s(v_1) = (a \cdot s(v_2, v_1))^* \cdot 0$$

$$s(v_2, v_1) = 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1))$$

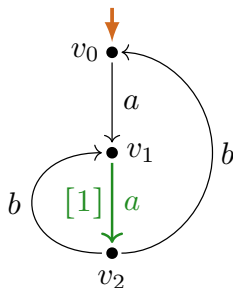
$$s(v_1, v_1) = 1$$

$$s(v_0, v_1) = 0^* \cdot a \cdot s(v_1, v_1)$$

$$= 0^* \cdot a \cdot 1$$

$$= \text{Mil}^- a$$

Readback from layered LEE-witness (example)



layered
LEE-witness

$$s(v_0) = 0^* \cdot a \cdot s(v_1)$$

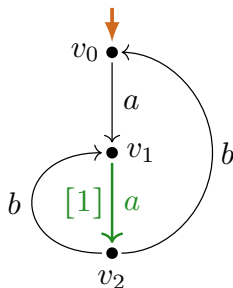
$$s(v_1) = (a \cdot s(v_2, v_1))^* \cdot 0$$

$$\begin{aligned} s(v_2, v_1) &= 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1)) \\ &=_{\text{Mil}} 0^* \cdot (b \cdot 1 + b \cdot a) \end{aligned}$$

$$s(v_1, v_1) = 1$$

$$\begin{aligned} s(v_0, v_1) &= 0^* \cdot a \cdot s(v_1, v_1) \\ &= 0^* \cdot a \cdot 1 \\ &=_{\text{Mil}} a \end{aligned}$$

Readback from layered LEE-witness (example)



layered
LEE-witness

$$s(v_0) = 0^* \cdot a \cdot s(v_1)$$

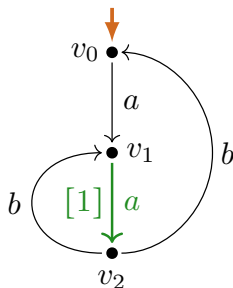
$$s(v_1) = (a \cdot s(v_2, v_1))^* \cdot 0$$

$$\begin{aligned} s(v_2, v_1) &= 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1)) \\ &=_{\text{Mil}} 0^* \cdot (b \cdot 1 + b \cdot a) \\ &=_{\text{Mil}} b + b \cdot a \end{aligned}$$

$$s(v_1, v_1) = 1$$

$$\begin{aligned} s(v_0, v_1) &= 0^* \cdot a \cdot s(v_1, v_1) \\ &= 0^* \cdot a \cdot 1 \\ &=_{\text{Mil}} a \end{aligned}$$

Readback from layered LEE-witness (example)



layered
LEE-witness

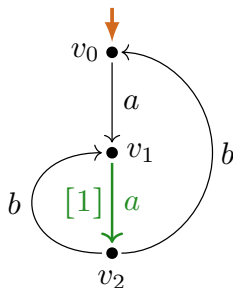
$$s(v_0) = 0^* \cdot a \cdot s(v_1)$$

$$\begin{aligned} s(v_1) &= (a \cdot s(v_2, v_1))^* \cdot 0 \\ &=_{\text{Mil}^-} (a \cdot (b + b \cdot a))^* \cdot 0 \end{aligned}$$

$$\begin{aligned} s(v_2, v_1) &= 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1)) \\ &=_{\text{Mil}^-} 0^* \cdot (b \cdot 1 + b \cdot a) \\ &=_{\text{Mil}^-} b + b \cdot a \end{aligned}$$

$$\begin{aligned} s(v_1, v_1) &= 1 \\ s(v_0, v_1) &= 0^* \cdot a \cdot s(v_1, v_1) \\ &= 0^* \cdot a \cdot 1 \\ &=_{\text{Mil}^-} a \end{aligned}$$

Readback from layered LEE-witness (example)



layered
LEE-witness

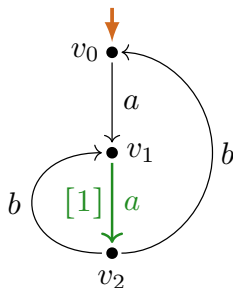
$$\begin{aligned} s(v_0) &= 0^* \cdot a \cdot s(v_1) \\ &=_{\text{Mil}^-} a \cdot s(v_1) \end{aligned}$$

$$\begin{aligned} s(v_1) &= (a \cdot s(v_2, v_1))^* \cdot 0 \\ &=_{\text{Mil}^-} (a \cdot (b + b \cdot a))^* \cdot 0 \end{aligned}$$

$$\begin{aligned} s(v_2, v_1) &= 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1)) \\ &=_{\text{Mil}^-} 0^* \cdot (b \cdot 1 + b \cdot a) \\ &=_{\text{Mil}^-} b + b \cdot a \end{aligned}$$

$$\begin{aligned} s(v_1, v_1) &= 1 \\ s(v_0, v_1) &= 0^* \cdot a \cdot s(v_1, v_1) \\ &= 0^* \cdot a \cdot 1 \\ &=_{\text{Mil}^-} a \end{aligned}$$

Readback from layered LEE-witness (example)



layered
LEE-witness

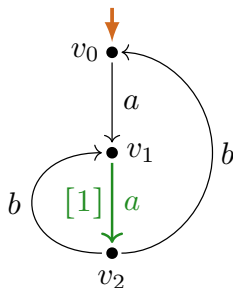
$$\begin{aligned} s(v_0) &= 0^* \cdot a \cdot s(v_1) \\ &=_{\text{Mil}^-} a \cdot s(v_1) \\ &=_{\text{Mil}^-} a \cdot (a \cdot (b + b \cdot a))^* \cdot 0 \end{aligned}$$

$$\begin{aligned} s(v_1) &= (a \cdot s(v_2, v_1))^* \cdot 0 \\ &=_{\text{Mil}^-} (a \cdot (b + b \cdot a))^* \cdot 0 \end{aligned}$$

$$\begin{aligned} s(v_2, v_1) &= 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1)) \\ &=_{\text{Mil}^-} 0^* \cdot (b \cdot 1 + b \cdot a) \\ &=_{\text{Mil}^-} b + b \cdot a \end{aligned}$$

$$\begin{aligned} s(v_1, v_1) &= 1 \\ s(v_0, v_1) &= 0^* \cdot a \cdot s(v_1, v_1) \\ &= 0^* \cdot a \cdot 1 \\ &=_{\text{Mil}^-} a \end{aligned}$$

Readback from layered LEE-witness (example)

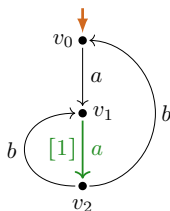


layered
LEE-witness

$$\begin{aligned}
 s(v_0) &= 0^* \cdot a \cdot s(v_1) \\
 &=_{\text{Mil}^-} a \cdot s(v_1) \\
 &=_{\text{Mil}^-} a \cdot \left(a \cdot (b + b \cdot a) \right)^* \cdot 0 \in \text{RExp}^{(*/+)} \\
 s(v_1) &= \left(a \cdot s(v_2, v_1) \right)^* \cdot 0 \\
 &=_{\text{Mil}^-} \left(a \cdot (b + b \cdot a) \right)^* \cdot 0 \\
 s(v_2, v_1) &= 0^* \cdot \left(b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1) \right) \\
 &=_{\text{Mil}^-} 0^* \cdot \left(b \cdot 1 + b \cdot a \right) \\
 &=_{\text{Mil}^-} b + b \cdot a \\
 s(v_1, v_1) &= 1 \\
 s(v_0, v_1) &= 0^* \cdot a \cdot s(v_1, v_1) \\
 &= 0^* \cdot a \cdot 1 \\
 &=_{\text{Mil}^-} a
 \end{aligned}$$

Readback

Readback steps reverse P -defined transitions up to $=_{Mil^-}$ and $\leftrightarrow$.



layered
LEE-witness

$$\begin{aligned}
 s(v_0) &= 0^* \cdot a \cdot s(v_1) \\
 &=_{Mil^-} a \cdot s(v_1) \\
 &=_{Mil^-} a \cdot (a \cdot (b + b \cdot a))^* \cdot 0 \in REp^{(*/+)} \\
 s(v_1) &= (a \cdot s(v_2, v_1))^* \cdot 0 \\
 &=_{Mil^-} (a \cdot (b + b \cdot a))^* \cdot 0 \\
 s(v_2, v_1) &= 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1)) \\
 &=_{Mil^-} 0^* \cdot (b \cdot 1 + b \cdot a) \\
 &=_{Mil^-} b + b \cdot a \\
 s(v_1, v_1) &= 1 \\
 s(v_0, v_1) &= 0^* \cdot a \cdot s(v_1, v_1) \\
 &= 0^* \cdot a \cdot 1 \\
 &=_{Mil^-} a
 \end{aligned}$$

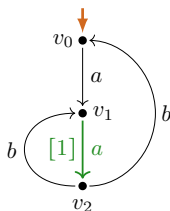
Readback

Lemma ($\sim$ G-Fokkink, 2020)

Process graphs with LLEE are $\llbracket \cdot \rrbracket_P$ -expressible by $(*/\dagger)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/\dagger)} (G \Leftrightarrow P(uf)).$$

Proof: Readback steps reverse P -defined transitions up to $=_{\text{Mil}^-}$ and $\Leftrightarrow$.



layered
LEE-witness

$$\begin{aligned} s(v_0) &= 0^* \cdot a \cdot s(v_1) \\ &=_{\text{Mil}^-} a \cdot s(v_1) \\ &=_{\text{Mil}^-} a \cdot (a \cdot (b + b \cdot a))^* \cdot 0 \in \text{RExp}^{(*/\dagger)} \\ s(v_1) &= (a \cdot s(v_2, v_1))^* \cdot 0 \\ &=_{\text{Mil}^-} (a \cdot (b + b \cdot a))^* \cdot 0 \\ s(v_2, v_1) &= 0^* \cdot (b \cdot s(v_1, v_1) + b \cdot s(v_0, v_1)) \\ &=_{\text{Mil}^-} 0^* \cdot (b \cdot 1 + b \cdot a) \\ &=_{\text{Mil}^-} b + b \cdot a \\ s(v_1, v_1) &= 1 \\ s(v_0, v_1) &= 0^* \cdot a \cdot s(v_1, v_1) \\ &= 0^* \cdot a \cdot 1 \\ &=_{\text{Mil}^-} a \end{aligned}$$

Readback (refined)

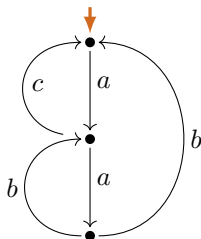
Statement

Process graphs with LLEE are P -expressible by $(*/\perp)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/\perp)} (G \sim P(uf)).$$

Proof: Refine readback steps to precisely reverse P -defined transitions!

G_2



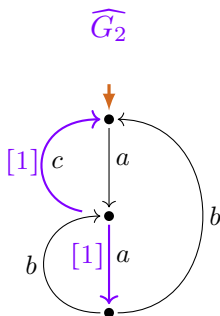
Readback (refined)

Statement

Process graphs with LLEE are P -expressible by $(*/\perp)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/\perp)} (G \sim P(uf)).$$

Proof: Refine readback steps to precisely reverse P -defined transitions!



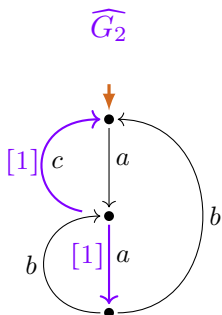
Readback (refined)

Statement

Process graphs with LLEE are P -expressible by $(*/\perp)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/\perp)} (G \sim P(uf)).$$

Proof: Refine readback steps to precisely reverse P -defined transitions!



$$P((1 \cdot (\quad)^*) \cdot 0)$$

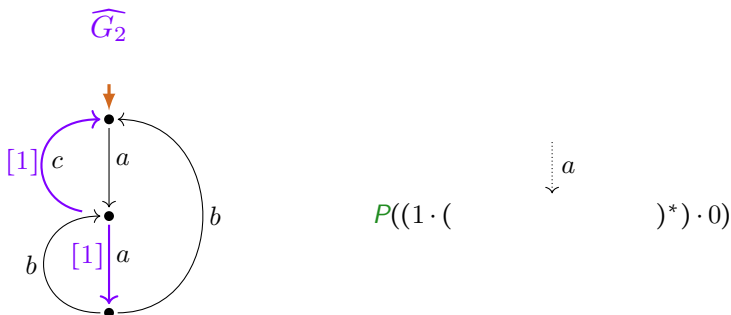
Readback (refined)

Statement

Process graphs with LLEE are P -expressible by $(*/\perp)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/\perp)} (G \sim P(uf)).$$

Proof: Refine readback steps to precisely reverse P -defined transitions!



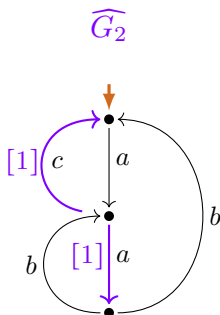
Readback (refined)

Statement

Process graphs with LLEE are P -expressible by $(*/\perp)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/\perp)} (G \sim P(uf)).$$

Proof: Refine readback steps to precisely reverse P -defined transitions!



$$\begin{array}{c}
 P(((1 \cdot a) \cdot ()^*) \cdot 0) \\
 \downarrow a \\
 P((1 \cdot ()^*) \cdot 0)
 \end{array}$$

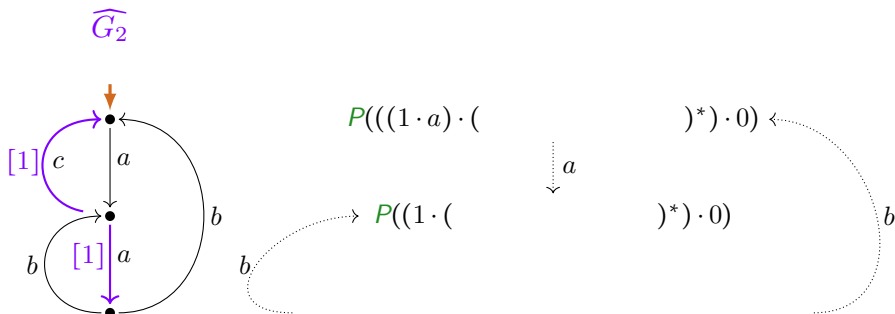
Readback (refined)

Statement

Process graphs with LLEE are P -expressible by $(*/\perp)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/\perp)} (G \sim P(uf)).$$

Proof: Refine readback steps to precisely reverse P -defined transitions!



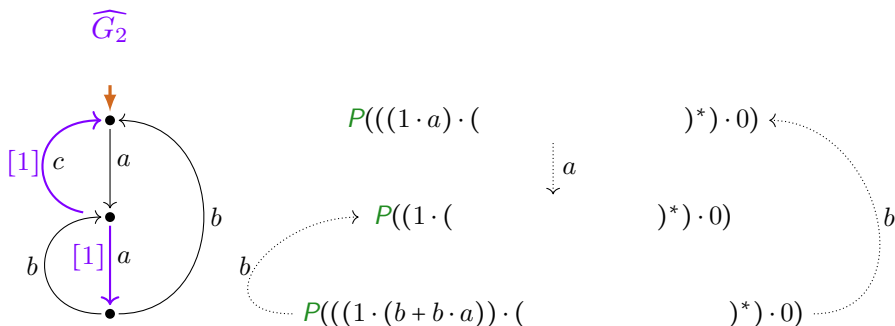
Readback (refined)

Statement

Process graphs with LLEE are P -expressible by $(*/\perp)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/\perp)} (G \sim P(uf)).$$

Proof: Refine readback steps to precisely reverse P -defined transitions!



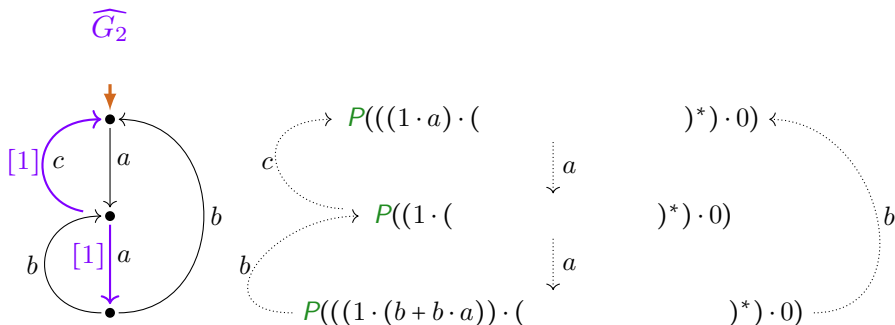
Readback (refined)

Statement

Process graphs with LLEE are P -expressible by $(*/\perp)$ reg. expressions:

$$LLEE(G) \implies \exists uf \in RExp^{(*/\perp)} (G \sim P(uf)).$$

Proof: Refine readback steps to precisely reverse P -defined transitions!



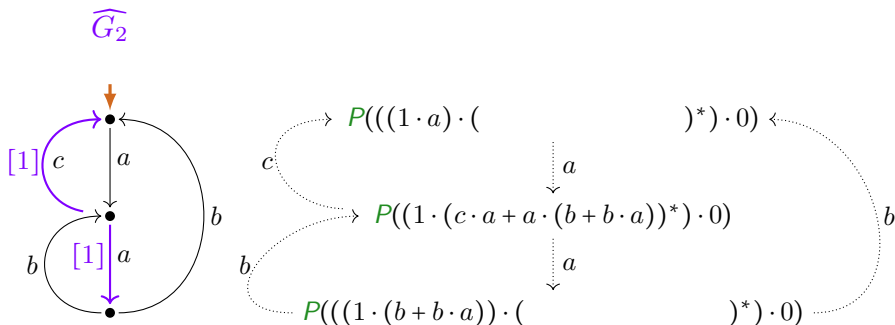
Readback (refined)

Statement

Process graphs with LLEE are P -expressible by $(*/\perp)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/\perp)} (G \sim P(uf)).$$

Proof: Refine readback steps to precisely reverse P -defined transitions!



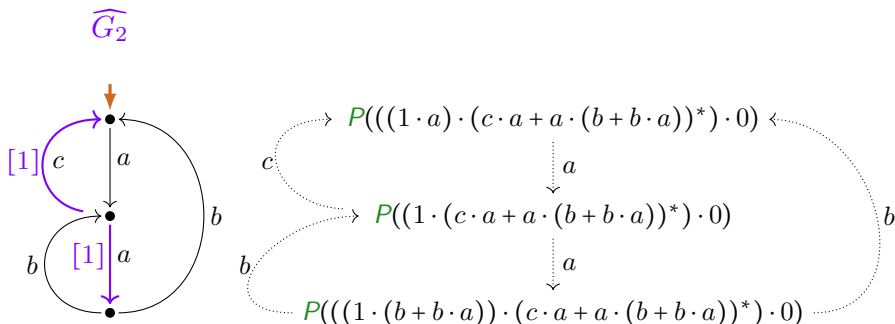
Readback (refined)

Statement

Process graphs with LLEE are P -expressible by $(*/1)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/+)} (G \sim P(uf)).$$

Proof: Refine readback steps to precisely reverse P -defined transitions!



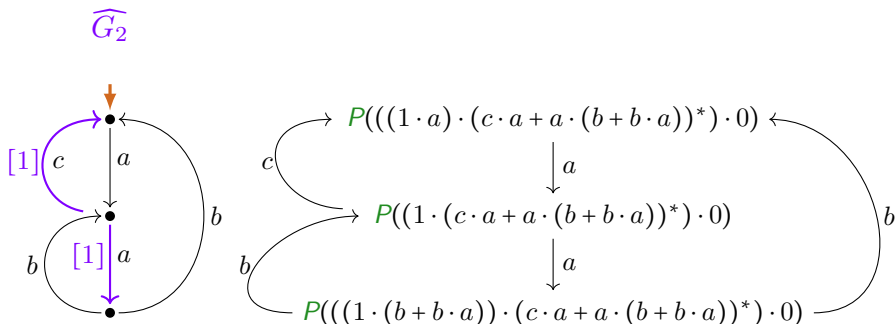
Readback (refined)

Statement

Process graphs with LLEE are P -expressible by $(*/\dagger)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/\dagger)} (G \sim P(uf)).$$

Proof: Refine readback steps to precisely reverse P -defined transitions!



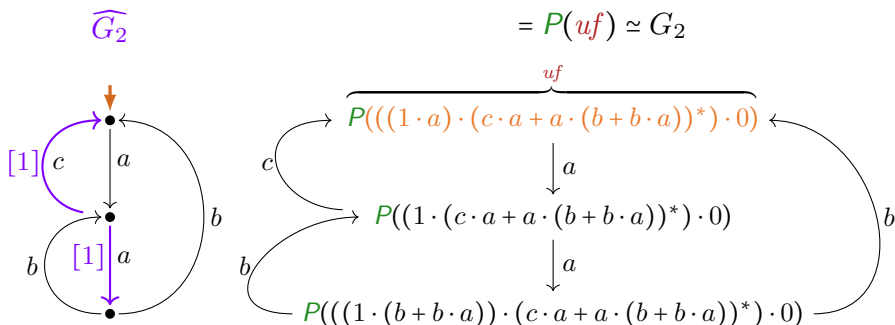
Readback (refined)

Statement

Process graphs with LLEE are P -expressible by $(*/\dagger)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/\dagger)} (G \sim P(uf)).$$

Proof: Refine readback steps to precisely reverse P -defined transitions!



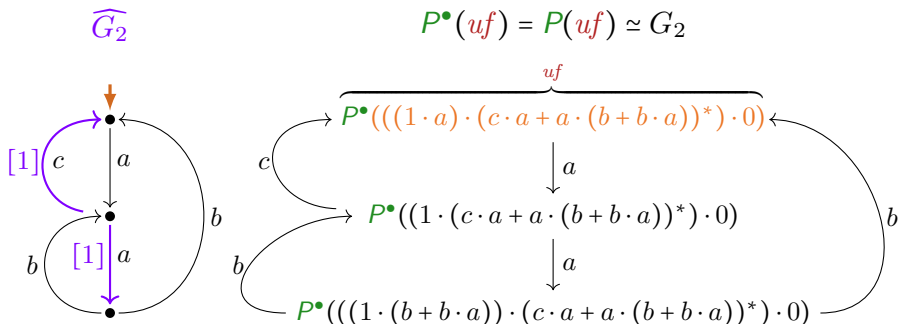
Readback (refined)

Lemma (*TERMGRAPH '24/'26*)

Process graphs with LLEE are $P^\bullet$ -expressible by $(*/1)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/+)} (G \sim P^\bullet(uf)).$$

Proof: Refine readback steps to precisely reverse $P^\bullet$ -defined transitions!



Characterising $P^\bullet$ -expressibility

Theorem ($\text{LLEE} \stackrel{\wedge}{=} \text{image of } P^\bullet |_{\text{RExp}(*/\perp)}$)

A process graph G is $P^\bullet$ -expressible by an $(*/\perp)$ regular expression
 $\iff G$ is finite, and G satisfies LLEE.

Characterising $P^\bullet$ -expressibility

Theorem ($LLEE \triangleq \text{image of } P^\bullet |_{RExp(*/\perp)}$)

A process graph G is $P^\bullet$ -expressible by an $(*/\perp)$ regular expression
 $\iff G$ is finite, and G satisfies LLEE.

Corollary (P -expressibility by $(*/\perp)$ expr's)

For every process graph G , TFAE:

- (i) G is $\llbracket \cdot \rrbracket_P$ -expressible by an $(*/\perp)$ regular expression.
- (ii) The bisimulation collapse of G is finite, and satisfies LLEE.
- (iii) The bisimulation collapse of G
 is $P^\bullet$ -expressible by a $(*/\perp)$ regular expression.

Characterising $P^\bullet$ -expressibility

Theorem ($\text{LLEE} \stackrel{\wedge}{=} \text{image of } P^\bullet |_{\text{RExp}(*/\perp)}$)

A process graph G is $P^\bullet$ -expressible by an $(*/\perp)$ regular expression
 $\iff G$ is finite, and G satisfies LLEE.

Corollary (P -expressibility by $(*/\perp)$ expr's)

For every process graph G , TFAE:

- (i) G is $\llbracket \cdot \rrbracket_P$ -expressible by an $(*/\perp)$ regular expression.
- (ii) The bisimulation collapse of G is finite, and satisfies LLEE.
- (iii) The bisimulation collapse of G
 is $P^\bullet$ -expressible by a $(*/\perp)$ regular expression.

Theorem ($1\text{-LLEE} \stackrel{\wedge}{=} \text{image of } P^\bullet$)

A process graph G is $P^\bullet$ -expressible by a regular expression
 $\iff G$ is finite, and G satisfies 1-LLEE.

Summary

- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility question)
 - ▶ axioms for $\llbracket \cdot \rrbracket_P$ -identity ($\leadsto$ completeness question)
- ▶ Loop existence and elimination (LEE), layered form (LLEE)
- ▶ Small steps for hard questions
 - ▶ reduction steps for well-behaved specifications under bisimilarity
 - ▶ expressibility is decidable
 - ▶ collapse steps for LLEE graphs under bisimilarity
 - ▶ completeness for $(*/\perp)$ expressions
 - ▶ collapse steps, as far as possible, for 1-LLEE graphs under bisimilarity
 - ▶ counterexample to collapse, but leads to completeness
- ▶ Interpretation–readback correspondences
 - ▶ LEE/1-LLEE characterise restricted to $(*/\perp)$ /unrestricted image of $P^\bullet$
 - ▶ where $P^\bullet$ a sharing-increased compact refinement of P

Summary

- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility question)
 - ▶ axioms for $\llbracket \cdot \rrbracket_P$ -identity ($\leadsto$ completeness question)
- ▶ Loop existence and elimination (LEE), layered form (LLEE)
- ▶ Small steps for hard questions
 - ▶ reduction steps for well-behaved specifications under bisimilarity
 - ▶ expressibility is decidable
 - ▶ collapse steps for LLEE graphs under bisimilarity
 - ▶ completeness for $(*/\perp)$ expressions
 - ▶ collapse steps, as far as possible, for 1-LLEE graphs under bisimilarity
 - ▶ counterexample to collapse, but leads to completeness
- ▶ Interpretation–readback correspondences
 - ▶ LEE/1-LEE characterise restricted to $(*/\perp)$ /unrestricted image of $P^\bullet$
 - ▶ where $P^\bullet$ a sharing-increased compact refinement of P

Next aims, and refined questions

Expressibility problems (for $P^\bullet$ and $[\![\cdot]\!]_P$)

A1: graph structure of regular expression processes (LEE/1-LEE)

- ▶ $LLEE \hat{=} \text{im}(P^\bullet)((*/\perp))$ $1\text{-}LLEE \hat{=} \text{im}(P^\bullet)$ (TERMGRAPH'26)
- ▶ $LLEE \hat{=} \text{co-reducible}^+$ graphs ($\Leftarrow$ Laarakker/Kappè, TERMGRAPH'26)

Next aims, and refined questions

Expressibility problems (for $P^\bullet$ and $\llbracket \cdot \rrbracket_P$)

A1: graph structure of regular expression processes (LEE/1-LEE)

- ▶ $LLEE \hat{=} \text{im}(P^\bullet)((*/\perp))$ $1\text{-}LLEE \hat{=} \text{im}(P^\bullet)$ (TERMGRAPH'26)
- ▶ $LLEE \hat{=} \text{co-reducible}^+$ graphs ($\Leftarrow$ Laarakker/Kappè, TERMGRAPH'26)

A2: LEE/ $P^\bullet$ -Expressibility is decidable in $\leq$ cubic time.

$\Leftarrow$ confluence of loop elimination with pruning steps (IWC'26),
which also entails $LLEE = LEE$.

Next aims, and refined questions

Expressibility problems (for $P^\bullet$ and $\llbracket \cdot \rrbracket_P$)

A1: graph structure of regular expression processes (LEE/1-LEE)

- ▶ $LLEE \hat{=} \text{im}(P^\bullet)((*/\perp))$ 1-LLEE $\hat{=} \text{im}(P^\bullet)$ (TERMGRAPH'26)
- ▶ $LLEE \hat{=} \text{co-reducible}^+$ graphs ($\Leftarrow$ Laarakker/Kappè, TERMGRAPH'26)

A2: LEE/ $P^\bullet$ -Expressibility is decidable in $\leq$ cubic time.

$\Leftarrow$ confluence of loop elimination with pruning steps (IWC'26),
which also entails $LLEE = LEE$.

Q1: Is 1-LLEE/ $P^\bullet$ -expressibility decidable in polynomial time?

Q2: Is $\llbracket \cdot \rrbracket_P$ -expressibility by a regular expression, for a finite process graph,
decidable in polynomial time/fixed-parameter tractable time?

Next aims, and refined questions

Expressibility problems (for $P^\bullet$ and $[\![\cdot]\!]_P$)

A1: graph structure of regular expression processes (LEE/1-LEE)

- ▶ $LLEE \hat{=} \text{im}(P^\bullet)((*/\perp))$ 1-LLEE $\hat{=} \text{im}(P^\bullet)$ (TERMGRAPH'26)
- ▶ $LLEE \hat{=} \text{co-reducible}^+$ graphs ($\Leftarrow$ Laarakker/Kappè, TERMGRAPH'26)

A2: LEE/ $P^\bullet$ -Expressibility is decidable in $\leq$ cubic time.

$\Leftarrow$ confluence of loop elimination with pruning steps (IWC'26),
which also entails $LLEE = LEE$.

Q1: Is 1-LLEE/ $P^\bullet$ -expressibility decidable in polynomial time?

Q2: Is $[\![\cdot]\!]_P$ -expressibility by a regular expression, for a finite process graph,
decidable in polynomial time/fixed-parameter tractable time?

Completeness problem, solution:

A3: motivation of crystallization

A4: details of crystallization procedure,
and completeness of Milner's proof system

TERMGRAPH / IWC Talks

TERMGRAPH (tomorrow, 19 July)

CG: *Towards a Characterisation of the Graph Structure of Process Interpretations of Regular Expressions*

Ext. Abstract <https://clegra.github.io/lf/pi-struc-TG-26.pdf>

- ▶ $L\bar{E}E \stackrel{\wedge}{=} \text{image of } P^\bullet |_{RExp(*/\pm)}$
- ▶ $1-L\bar{E}E \stackrel{\wedge}{=} \text{image of } P^\bullet$

TERMGRAPH / IWC Talks

TERMGRAPH (tomorrow, 19 July)

CG: *Towards a Characterisation of the Graph Structure of Process Interpretations of Regular Expressions*

Ext. Abstract <https://clegra.github.io/lf/pi-struc-TG-26.pdf>

- ▶ $\text{LEE} \triangleq \text{image of } P^\bullet |_{\text{RExp}(* / \pm)}$
- ▶ $1\text{-LEE} \triangleq \text{image of } P^\bullet$

IWC (Friday, 24 July)

CG: *Loop Elimination in Process Graphs is Confluent When Pruning Steps are Added*

Ext. Abstract <https://clegra.github.io/lf/le-conf-IWC-26.pdf>

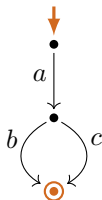
- ▶ critical-pair like analysis of loop elimination steps $\longrightarrow_{\text{elim}}$
- ▶ $\longrightarrow_{\text{elim}}$ forks are joinable by $\longrightarrow_{\text{elim}}$ steps and prune steps $\longrightarrow_{\text{prune}}$

Resources

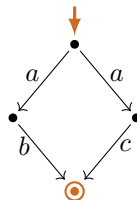
- ▶ Slides on clegra.github.io
 - ▶ slides: [.../1f/IFIP-1.6-2026.pdf](https://clegra.github.io/1f/IFIP-1.6-2026.pdf)
- ▶ CG: Closing the Image of the Process Interpretation of 1-Free Regular Expressions Under Bisimulation Collapse
 - ▶ TERMGRAPH 2024, [extended abstract](#)
- ▶ CG: The Image of the Process Interpretation of Regular Expressions is Not Closed under Bisimulation Collapse
 - ▶ [arXiv:2303.08553](#), 2021/2023.
- ▶ CG: Milner's Proof System for Regular Expressions Modulo Bisimilarity is Complete
 - ▶ LICS 2022, [arXiv:2209.12188](#), [poster](#)
- ▶ CG, Wan Fokkink: A Complete Proof System for 1-Free Regular Expressions Modulo Bisimilarity
 - ▶ LICS 2020, [arXiv:2004.12740](#), [video on youtube](#)
- ▶ CG: Modeling Terms by Graphs with Structure Constraints
 - ▶ TERMGRAPH 2018, [EPTCS 288](#), [arXiv:1902.02010](#)

Process semantics equality $=_{\llbracket \cdot \rrbracket_P}$

- Fewer identities hold for $=_{\llbracket \cdot \rrbracket_P}$ than for $=_L$:



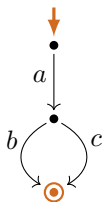
$$P(a \cdot (b + c))$$



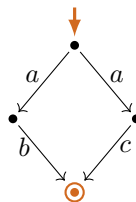
$$P(a \cdot b + a \cdot c)$$

Process semantics equality $=_{\llbracket \cdot \rrbracket_P}$

- Fewer identities hold for $=_{\llbracket \cdot \rrbracket_P}$ than for $=_L$:



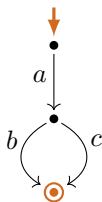
$$P(a \cdot (b + c))$$



$$P(a \cdot b + a \cdot c)$$

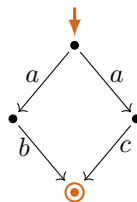
Process semantics equality $=_{\llbracket \cdot \rrbracket_P}$

- Fewer identities hold for $=_{\llbracket \cdot \rrbracket_P}$ than for $=_L$:



$$a \cdot (b + c)$$

$\neq_{\llbracket \cdot \rrbracket_P}$

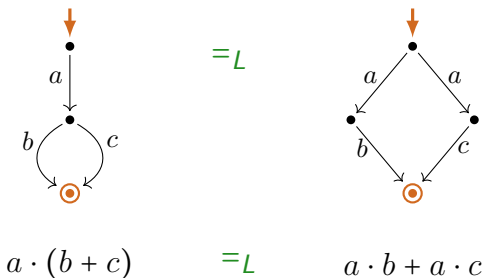


$$a \cdot b + a \cdot c$$

$\neq_{\llbracket \cdot \rrbracket_P}$

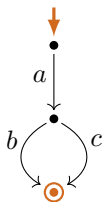
Process semantics equality $=_{\llbracket \cdot \rrbracket_P}$

- Fewer identities hold for $=_{\llbracket \cdot \rrbracket_P}$ than for $=_L$:



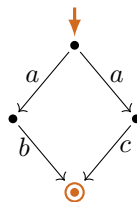
Process semantics equality $=_{\llbracket \cdot \rrbracket_P}$

► Fewer identities hold for $=_{\llbracket \cdot \rrbracket_P}$ than for $=_L$: $=_{\llbracket \cdot \rrbracket_P} \not\subseteq =_L$.



$a \cdot (b + c)$

$\neq_{\llbracket \cdot \rrbracket_P}$



$a \cdot b + a \cdot c$

$\neq_{\llbracket \cdot \rrbracket_P}$

Milner's proof system **Mil**

Axioms:

$$(A1) \quad e + (f + g) = (e + f) + g$$

$$(A2) \quad e + 0 = e$$

$$(A3) \quad e + f = f + e$$

$$(A4) \quad e + e = e$$

$$(A5) \quad e \cdot (f \cdot g) = (e \cdot f) \cdot g$$

$$(A6) \quad (e + f) \cdot g = e \cdot g + f \cdot g$$

$$(A7) \quad e = 1 \cdot e$$

$$(A8) \quad e = e \cdot 1$$

$$(A9) \quad 0 = 0 \cdot e$$

$$(A10) \quad e^* = 1 + e \cdot e^*$$

$$(A11) \quad e^* = (1 + e)^*$$

$$\text{But: } e \cdot (f + g) \neq e \cdot f + e \cdot g$$

$$\text{But: } e \cdot 0 \neq 0$$

Inference rules: rules of equational logic *plus*

$$\frac{e = f \cdot e + g}{e = f^* \cdot g} \text{RSP}^* \text{ (if } f \text{ does not terminate immediately)}$$

Milner's Completeness Question

Is **Mil** complete with respect to $=_{[\cdot]_P}$? (Does $e =_{[\cdot]_P} f \implies e =_{\text{Mil}} f$ hold?)

Milner's questions

(Q1) Completeness Question:

*Is the proof system Mil **complete** for $=_{\llbracket \cdot \rrbracket_P}$?*

(Q2) $\llbracket \cdot \rrbracket_P$ -Expressibility Question:

*What **structural property** characterizes
process graphs that are $\llbracket \cdot \rrbracket_P$ -**expressible** ?*

Milner's questions

(Q1) Completeness Question:

Is the proof system Mil complete for $=_{\llbracket \cdot \rrbracket_P}$?

(Q2) $\llbracket \cdot \rrbracket_P$ -Expressibility Question:

What structural property characterizes process graphs that are $\llbracket \cdot \rrbracket_P$ -expressible?

- ▶ is decidable (Baeten/Corradini/G, 2007)

Milner's questions

(Q1) Completeness Question:

Is the proof system Mil complete for $=_{\llbracket \cdot \rrbracket_P}$?

(Q2) $\llbracket \cdot \rrbracket_P$ -Expressibility Question:

What structural property characterizes process graphs that are $\llbracket \cdot \rrbracket_P$ -expressible?

- ▶ is decidable (Baeten/Corradini/G, 2007)
- ▶ partial new answer (G/Fokkink, 2020):
 - ▶ bisimulation collapse has loop existence & elimination property (LEE) if expressible by under-star-1-free regular expression

Milner's questions

(Q1) Completeness Question:

Is the proof system *Mil* *complete* for $=_{\llbracket \cdot \rrbracket_P}$?

- ▶ series of partial completeness results for:
 - ▶ exitless iterations (Fokkink, 1998)
 - ▶ with a stronger fixed-point rule (G, 2006)
 - ▶ under-star 1-free, and without 0 (Corradini/de Nicola/Labella, 2004)
 - ▶ with 0 but under-star-1-free (G/Fokkink, 2020)

(Q2) $\llbracket \cdot \rrbracket_P$ -Expressibility Question:

What *structural property* characterizes
process graphs that are $\llbracket \cdot \rrbracket_P$ -expressible ?

- ▶ is decidable (Baeten/Corradini/G, 2007)
- ▶ partial new answer (G/Fokkink, 2020):
 - ▶ bisimulation collapse has *loop existence & elimination property* (LEE)
if expressible by under-star-1-free regular expression

Milner's questions

(Q1) Completeness Question:

Is the proof system *Mil* *complete* for $=_{\llbracket \cdot \rrbracket_P}$?

- ▶ Yes! (G, 2022, proof summary, employing LEE and crystallization)
- ▶ series of partial completeness results for:
 - ▶ exitless iterations (Fokkink, 1998)
 - ▶ with a stronger fixed-point rule (G, 2006)
 - ▶ under-star 1-free, and without 0 (Corradini/de Nicola/Labella, 2004)
 - ▶ with 0 but under-star-1-free (G/Fokkink, 2020)

(Q2) $\llbracket \cdot \rrbracket_P$ -Expressibility Question:

What *structural property* characterizes process graphs that are $\llbracket \cdot \rrbracket_P$ -expressible?

- ▶ is decidable (Baeten/Corradini/G, 2007)
- ▶ partial new answer (G/Fokkink, 2020):
 - ▶ bisimulation collapse has loop existence & elimination property (LEE) if expressible by under-star-1-free regular expression

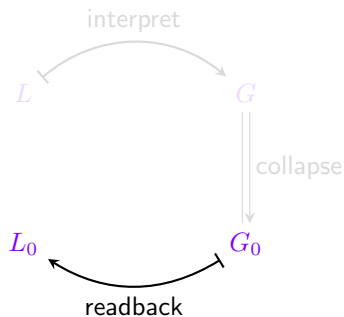
Question (Q2) specialized

(Q2)₀ P -Expressibility and P - $(*/\perp)$ -Expressibility:

What *structural property* characterizes:

- ▶ process graphs that are P -expressible ?
(... in the *image of P* ?)
- ▶ process graphs that are P -expressible by $(*/\perp)$ regular expressions?
(... in the *image of $(*/\perp)$ expressions under P* ?)

Readback



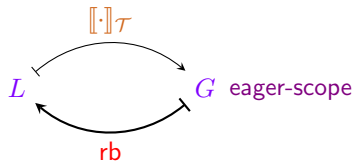
Readback

defined with property:



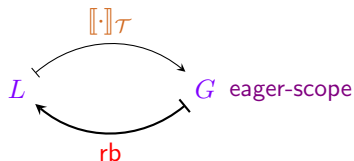
Readback

defined with property:



Readback

defined with property:



Theorem

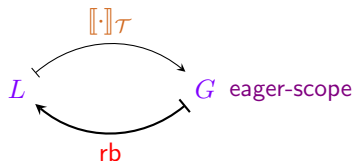
For all *eager-scope* λ -term-graphs G :

$$([[\cdot]]_{\mathcal{T}} \circ \text{rb})(G) \simeq G$$

The readback **rb** is a right-inverse of $[[\cdot]]_{\mathcal{T}}$ modulo isomorphism $\simeq$.

Readback

defined with property:



Theorem

For all *eager-scope* λ -term-graphs G :

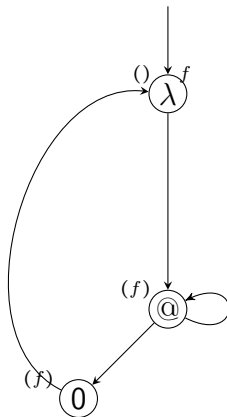
$$(\llbracket \cdot \rrbracket_{\mathcal{T}} \circ \text{rb})(G) \simeq G$$

The readback **rb** is a right-inverse of $\llbracket \cdot \rrbracket_{\mathcal{T}}$ modulo isomorphism $\simeq$.

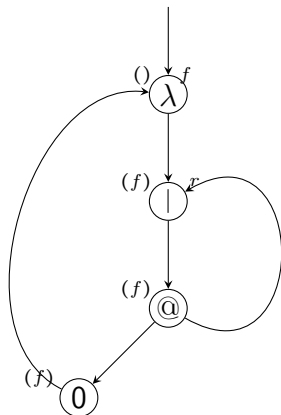
idea:

1. construct a spanning tree T of G
2. using local rules, in a bottom-up traversal of T synthesize $L = \text{rb}(G)$

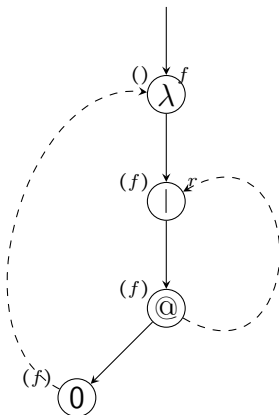
Readback: example (fix)



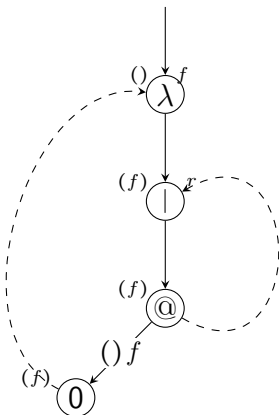
Readback: example (fix)



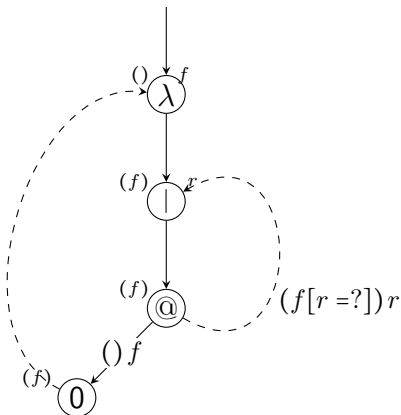
Readback: example (fix)



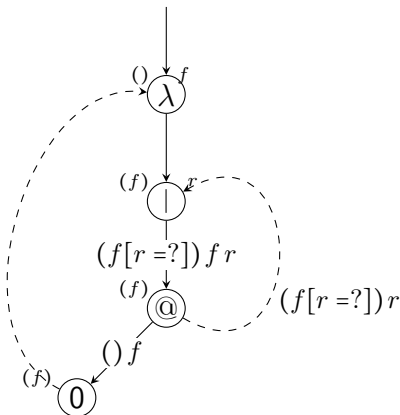
Readback: example (fix)



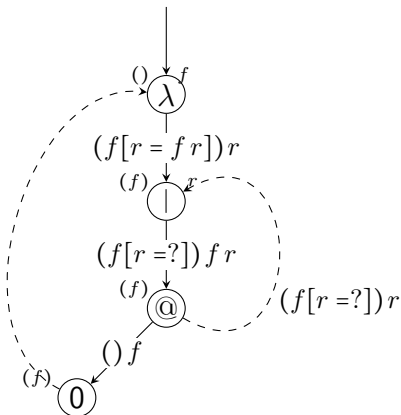
Readback: example (fix)



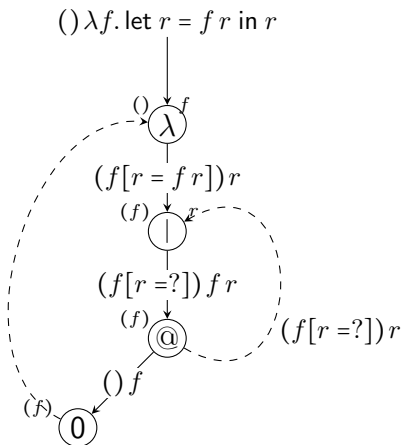
Readback: example (fix)



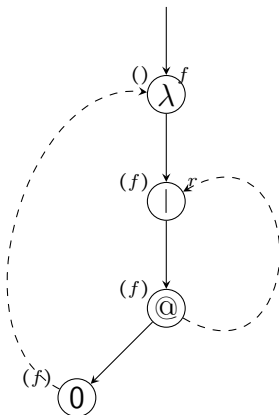
Readback: example (fix)



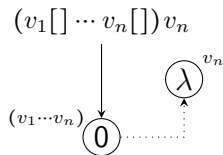
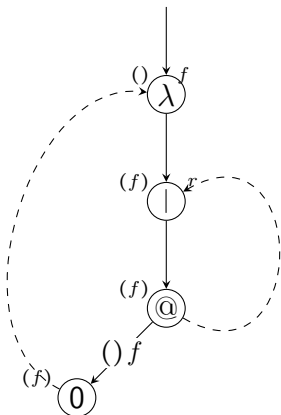
Readback: example (fix)



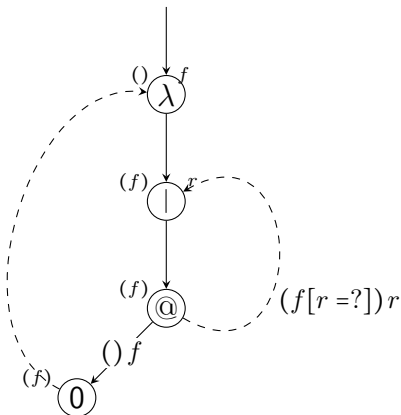
Readback: example (fix)



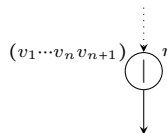
Readback: example (fix)



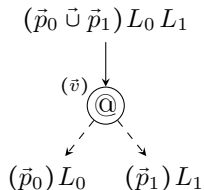
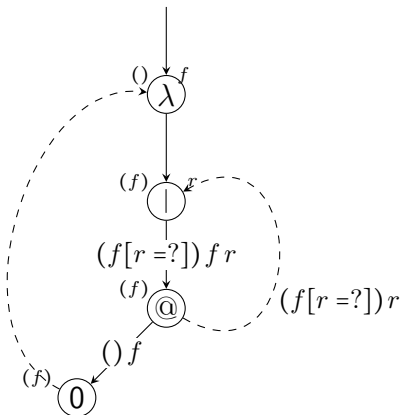
Readback: example (fix)



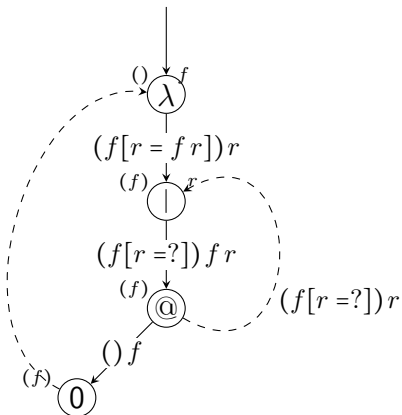
$(v_1[] \cdots v_n[] v_{n+1}[r = ?])r$



Readback: example (fix)



Readback: example (fix)



$$\begin{array}{c}
 (\vec{p} \ v_{n+1}[B, r = L]) r \\
 \downarrow \\
 (vs(\vec{p}) \ v_{n+1}) \downarrow r \\
 \downarrow \\
 (\vec{p} \ v_{n+1}[B, (r = ?)]) L
 \end{array}$$

